Plataforma Inteligente de Mantenimiento Predictivo

Estudiante

Daniel Camilo Ramos Garcia

Docente

Eliecer Montero Ojeda

Unidades Tecnológicas de Santander

Ingeniería de Sistemas

Bucaramanga

2026

1. Introducción

En el contexto industrial actual, la continuidad operativa de los equipos es fundamental para garantizar la eficiencia productiva y minimizar pérdidas económicas. Las fallas inesperadas en maquinaria pueden ocasionar interrupciones significativas en la producción y altos costos de reparación.

Tradicionalmente, las empresas han utilizado estrategias de mantenimiento correctivo o preventivo. Sin embargo, estas metodologías presentan limitaciones debido a que reaccionan ante fallas ya ocurridas o se basan en intervalos de tiempo predefinidos que no siempre reflejan el estado real de los equipos.

El **mantenimiento predictivo** surge como una alternativa avanzada que permite monitorear constantemente el estado de los equipos mediante sensores, analizar los datos obtenidos y anticipar posibles fallas antes de que ocurran.

El presente proyecto propone el diseño de una **Plataforma Inteligente de Mantenimiento Predictivo**, la cual permitirá gestionar activos industriales, registrar datos de sensores, detectar anomalías en el funcionamiento de los equipos y generar alertas que permitan planificar actividades de mantenimiento oportunamente.

El sistema será diseñado aplicando principios de **ingeniería de software y patrones de diseño**, con el objetivo de garantizar una arquitectura escalable, mantenible y reutilizable.

2. Objetivo General

Diseñar y desarrollar una plataforma web de mantenimiento predictivo aplicando patrones de diseño de software y buenas prácticas de arquitectura para demostrar un desarrollo modular y escalable.

3. Objetivos Específicos

- Diseñar el modelo conceptual del sistema mediante diagramas UML.
- Definir una arquitectura de software basada en capas.

- Implementar un backend que gestione los datos de sensores y activos industriales.
- Desarrollar una interfaz frontend que permita visualizar la información del sistema.
- Aplicar patrones de diseño para mejorar la organización y reutilización del código.
- Demostrar el funcionamiento de los patrones mediante diagramas UML específicos.

4. Descripción General del Sistema

La Plataforma Inteligente de Mantenimiento Predictivo permitirá monitorear equipos industriales mediante sensores que registran variables operativas como temperatura, vibración o presión.

El sistema almacenará estos datos y realizará un análisis para detectar anomalías que puedan indicar una posible falla en el equipo.

Cuando el sistema detecte una condición anormal, generará una alerta que permitirá a los responsables de mantenimiento tomar decisiones oportunas y programar actividades de reparación o inspección.

El sistema también permitirá gestionar técnicos y órdenes de mantenimiento para garantizar un seguimiento adecuado de las actividades realizadas.

5. Arquitectura General del Sistema

El sistema se desarrollará siguiendo una arquitectura cliente-servidor, compuesta por tres componentes principales:

- **Frontend:** interfaz de usuario web.
- **Backend:** API encargada de la lógica del sistema.
- **Base de datos:** almacenamiento de la información del sistema.

Tecnologías propuestas

Frontend

- React
- TypeScript
- TailwindCSS

Backend

- Node.js
- NestJS
- TypeScript

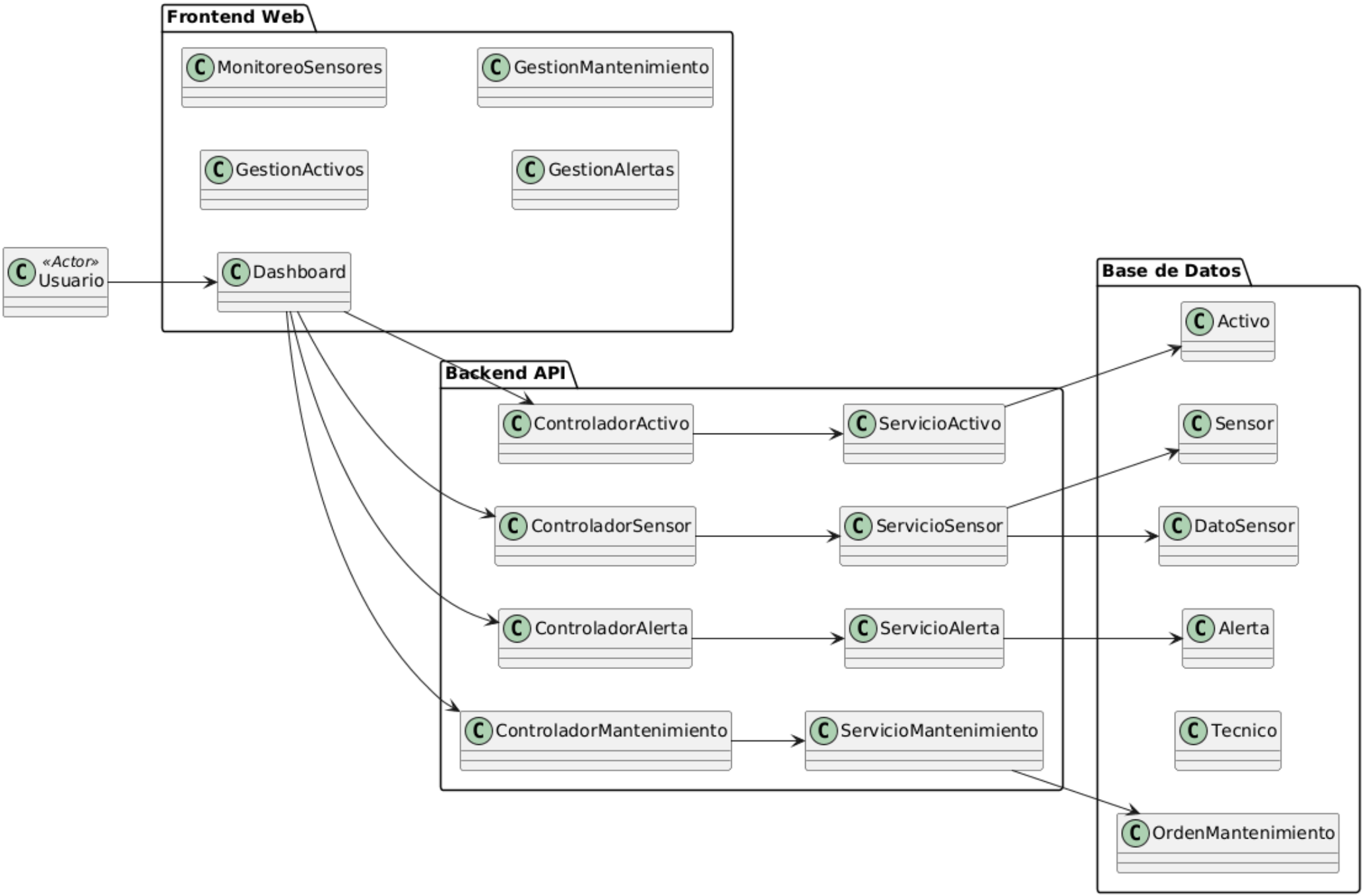
Base de datos

- PostgreSQL

6. Diagrama de Arquitectura del Sistema

Este diagrama muestra la interacción entre el usuario, el frontend, el backend y la base de datos.

Plataforma Inteligente de Mantenimiento Predictivo



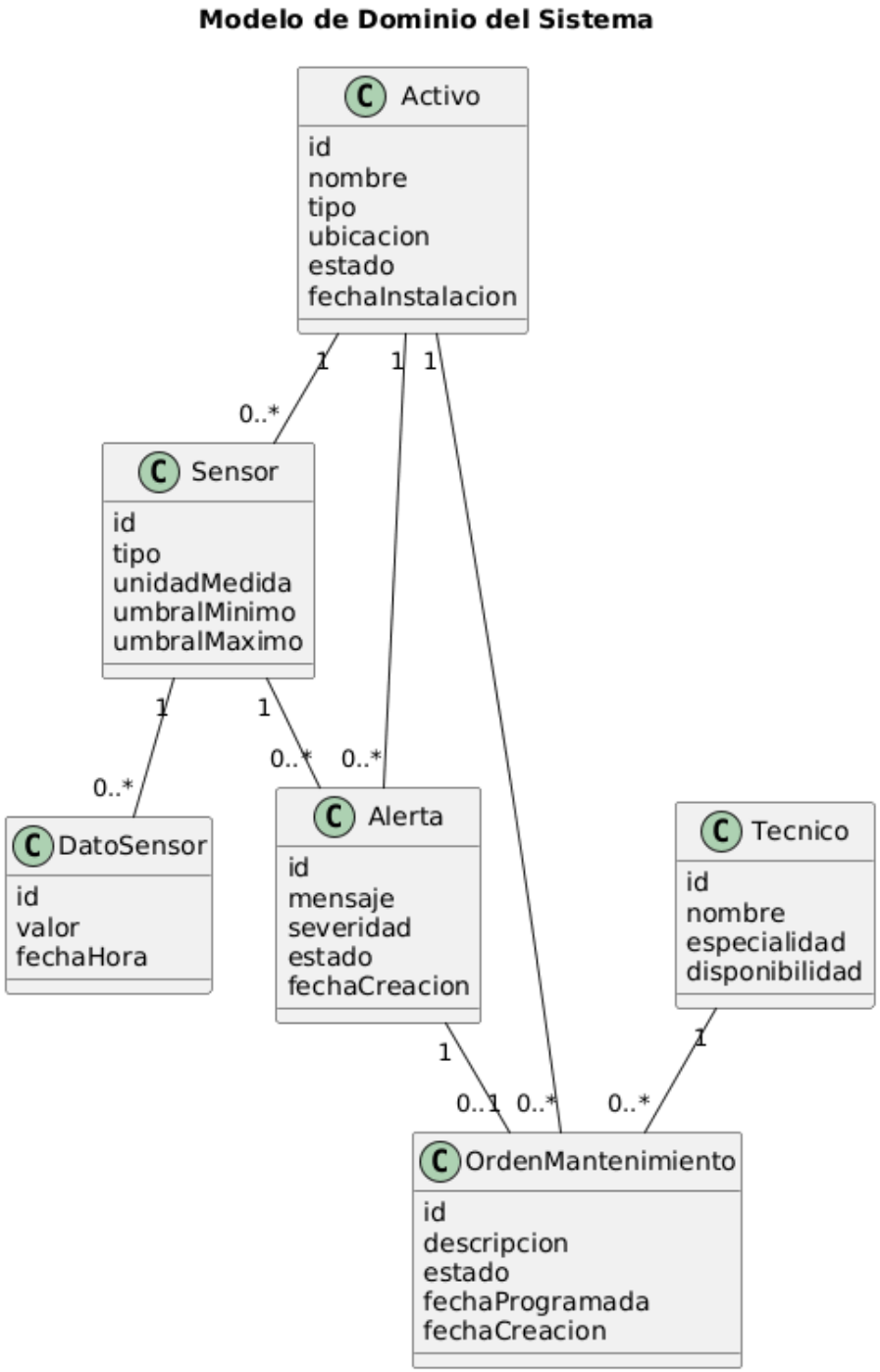
7. Modelo de Dominio del Sistema

El modelo de dominio representa las entidades principales del sistema y sus relaciones.

Las entidades principales del sistema son:

- Activo
- Sensor
- DatoSensor
- Alerta
- Técnico
- Orden de mantenimiento

8. Diagrama UML del Modelo de Dominio



9. Funcionamiento General del Sistema

El funcionamiento del sistema puede describirse mediante el siguiente flujo:

1. Los sensores instalados en los equipos registran datos operativos.
2. Los datos son enviados al sistema y almacenados en la base de datos.
3. El sistema analiza los datos recibidos.
4. Si se detecta un valor fuera de los rangos normales, se genera una alerta.
5. La alerta puede generar una orden de mantenimiento.
6. La orden es asignada a un técnico para su ejecución.

10. Funcionalidades del Sistema

- **Gestión de Activos**

  Permite registrar y administrar máquinas o equipos industriales.

- **Gestión de Sensores**

  Permite registrar sensores y asociarlos a los activos.

- **Monitoreo de Datos**

  Permite visualizar las mediciones enviadas por los sensores.

- **Gestión de Alertas**

  Permite identificar posibles fallas en los equipos.

- **Gestión de Mantenimiento**

  Permite generar y administrar órdenes de mantenimiento.

- **Gestión de Técnicos**

  Permite administrar el personal encargado de realizar el mantenimiento.

11. Estructura General del Proyecto

El sistema se organizará en dos componentes principales:

Backend

controllers

services

entities

repositories

factories

builders

modules

Frontend

frontend

components

pages

services

hooks

layouts

12. Diagrama de Casos de Uso

El diagrama de casos de uso muestra **los actores del sistema y las funcionalidades que pueden realizar.**

Actores del sistema

Administrador

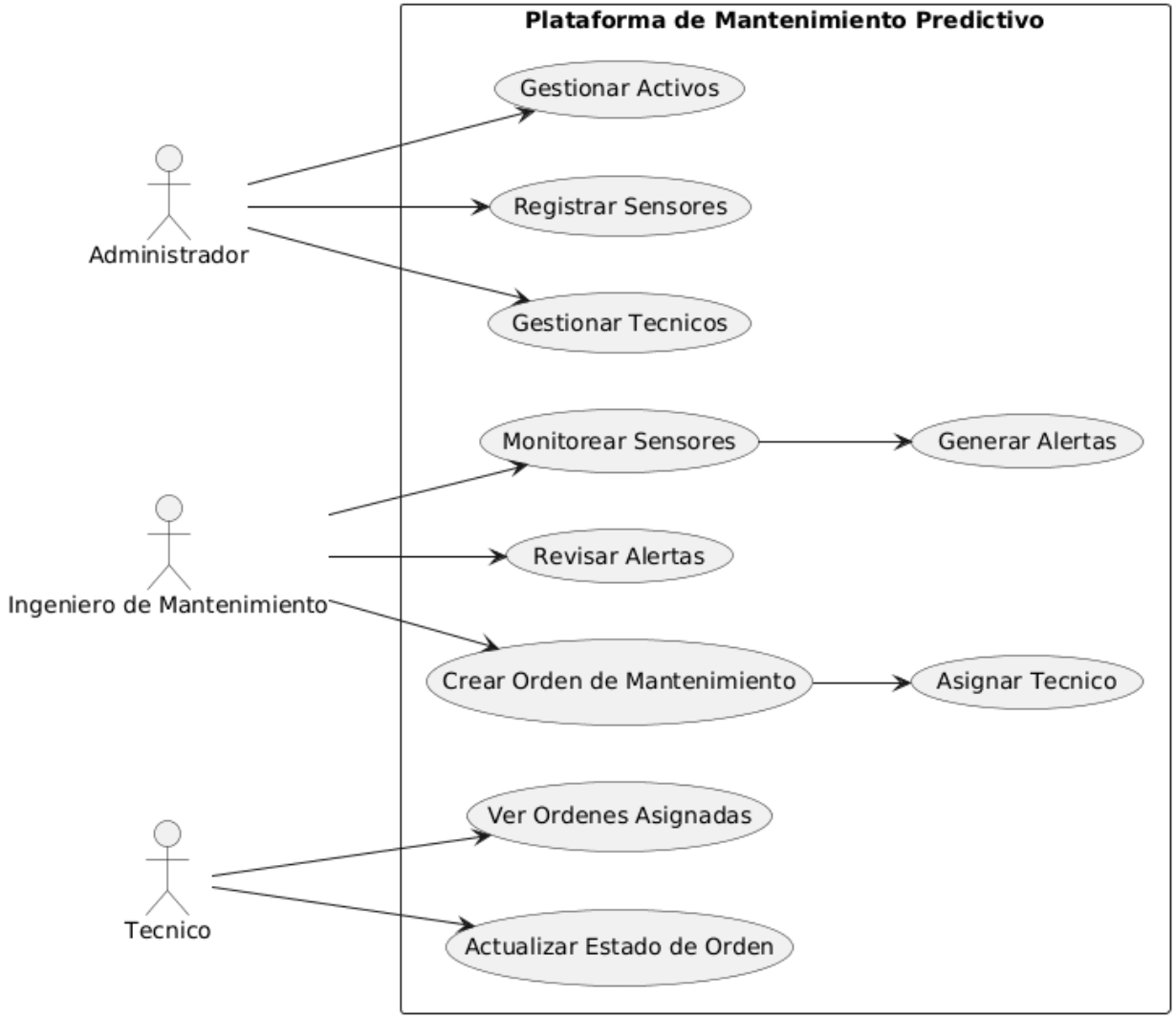
- Gestionar activos
- Registrar sensores
- Gestionar técnicos

Ingeniero de mantenimiento

- Ver monitoreo de sensores
- Revisar alertas
- Crear órdenes de mantenimiento

Técnico

- Ver órdenes asignadas
- Actualizar estado de mantenimiento

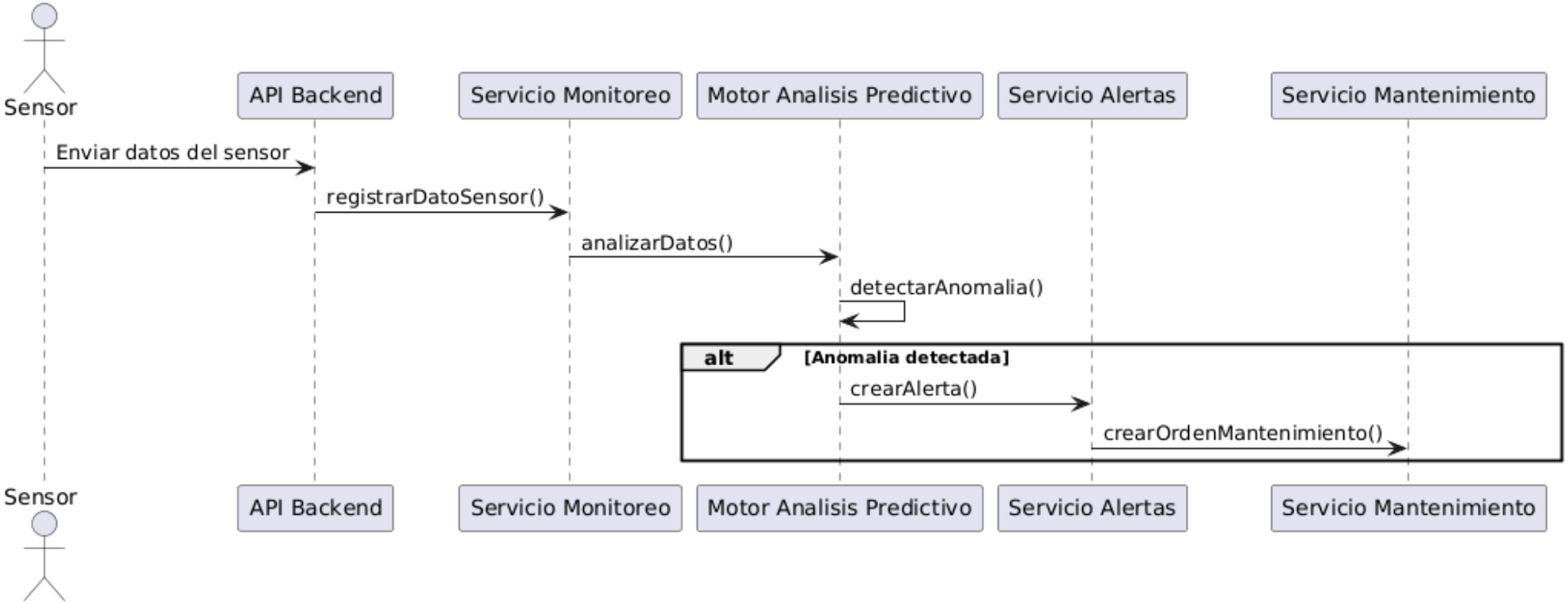


13. Diagrama de Secuencia del Sistema

Este diagrama muestra **cómo se genera una alerta a partir de datos de sensores.**

Escenario

1. Sensor envía datos
2. Backend guarda datos
3. Motor de análisis revisa valores
4. Si hay anomalía → genera alerta
5. Se crea orden de mantenimiento



Patrón Singleton

El patrón Singleton garantiza que una clase tenga una única instancia en todo el sistema y proporciona un punto global de acceso a dicha instancia.

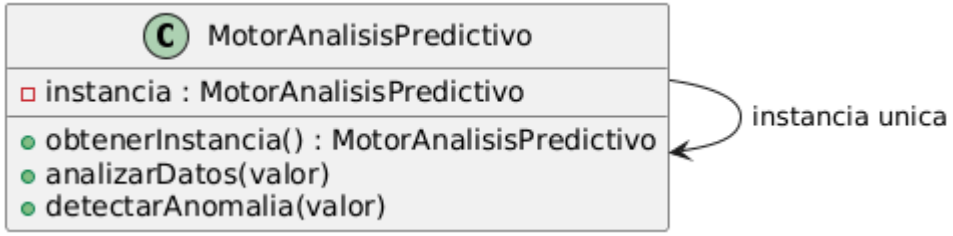
Aplicación en nuestro sistema

En la Plataforma de Mantenimiento Predictivo, el patrón Singleton se utilizará para implementar el:

Motor de Análisis Predictivo

Este componente será responsable de:

- analizar datos de sensores
- detectar anomalías
- generar alertas



Debe **existir una sola instancia**, porque:

- centraliza el análisis
- evita duplicación de lógica
- mantiene coherencia en el sistema

Implementación del Patrón Singleton

Se creó una clase llamada **MotorAnalisisPredictivo** que implementa la lógica del patrón Singleton.

- src/analysis/analysis-engine.singleton.ts

```
export class MotorAnalisisPredictivo {

  private static instancia: MotorAnalisisPredictivo;

  private constructor() {}

  public static obtenerInstancia(): MotorAnalisisPredictivo {

    if (!MotorAnalisisPredictivo.instancia) {
      MotorAnalisisPredictivo.instancia = new MotorAnalisisPredictivo();
    }

    return MotorAnalisisPredictivo.instancia;
  }

  analizarDatos(valor: number) {

    console.log("Analizando valor:", valor);

    return this.detectarAnomalia(valor);
  }

  detectarAnomalia(valor: number) {

    if (valor > 80) {
      return "ALERTA: posible falla detectada";
    }

    return "Valor normal";
  }

}
```

En esta implementación:

- El **constructor es privado**, evitando que otras clases creen instancias directamente.
- La variable **instancia** almacena el único objeto de la clase.
- El método **obtenerInstancia()** devuelve siempre la misma instancia del motor de análisis.

Integración del Singleton en el sistema

Para utilizar el motor de análisis se creó un servicio encargado de recibir los datos de los sensores y enviarlos al motor Singleton.

- src/sensors/sensors.service.ts

src > sensors > sensors.service.ts > SensorsService

```
1   import { Injectable } from '@nestjs/common';
2   import { MotorAnalisisPredictivo } from '../analysis/analysis-engine.singleton';
3
4   @Injectable()
5   export class SensorsService {
6
7     analizarValorSensor(valor: number) {
8
9       const motor = MotorAnalisisPredictivo.obtenerInstancia();
10
11      return motor.analizarDatos(valor);
12
13    }
14
15  }
```

Este servicio obtiene la instancia única del motor de análisis y procesa el valor recibido desde los sensores.

Controlador para el análisis de sensores

Se creó un controlador que expone un endpoint para enviar datos de sensores al sistema.

- src/sensors/sensors.controller.ts

src > sensors > sensors.controller.ts > SensorsController

```
1  import { Controller, Post, Body } from '@nestjs/common';
2  import { SensorsService } from './sensors.service';
3
4  @Controller('sensors')
5  export class SensorsController {
6
7    constructor(private readonly sensorsService: SensorsService) {}
8
9    @Post("analizar")
10   analizar(@Body() body: any) {
11
12     return this.sensorsService.analizarValorSensor(body.valor);
13
14   }
15
16 }
```

Este controlador permite enviar valores de sensores mediante una petición HTTP al backend.

Prueba del funcionamiento del patrón

Para validar el funcionamiento del sistema se utilizó **Postman** para enviar una petición al endpoint del sistema.

Endpoint utilizado

POST http://localhost:3000/sensors/analizar

Body de la petición

```
{
"valor": 90
}
```

Respuesta obtenida

ALERTA: posible falla detectada

Esto demuestra que el sistema recibe el valor del sensor, lo envía al Motor de Análisis Predictivo Singleton y retorna el resultado del análisis.

Unidades Tecnológicas de Santander

http://localhost:3000/sensors/analizar

Save

POST http://localhost:3000/sensors/analizar Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2 "valor": 90
3 }
```

Body Cookies Headers (7) Test Results 201 Created 27 ms 264 B Save Response

Pretty Raw Preview Visualize HTML

```
1 ALERTA: posible falla detectada
```

Flujo de funcionamiento del sistema

El flujo del proceso implementado es el siguiente:

Sensor → API REST → SensorsController → SensorsService → MotorAnalisisPredictivo (Singleton)

1. El sensor envía un valor al sistema.
2. El controlador recibe la petición HTTP.
3. El servicio procesa el valor recibido.
4. El motor de análisis Singleton evalúa el valor.
5. El sistema retorna el resultado del análisis.

Patrón Factory Method

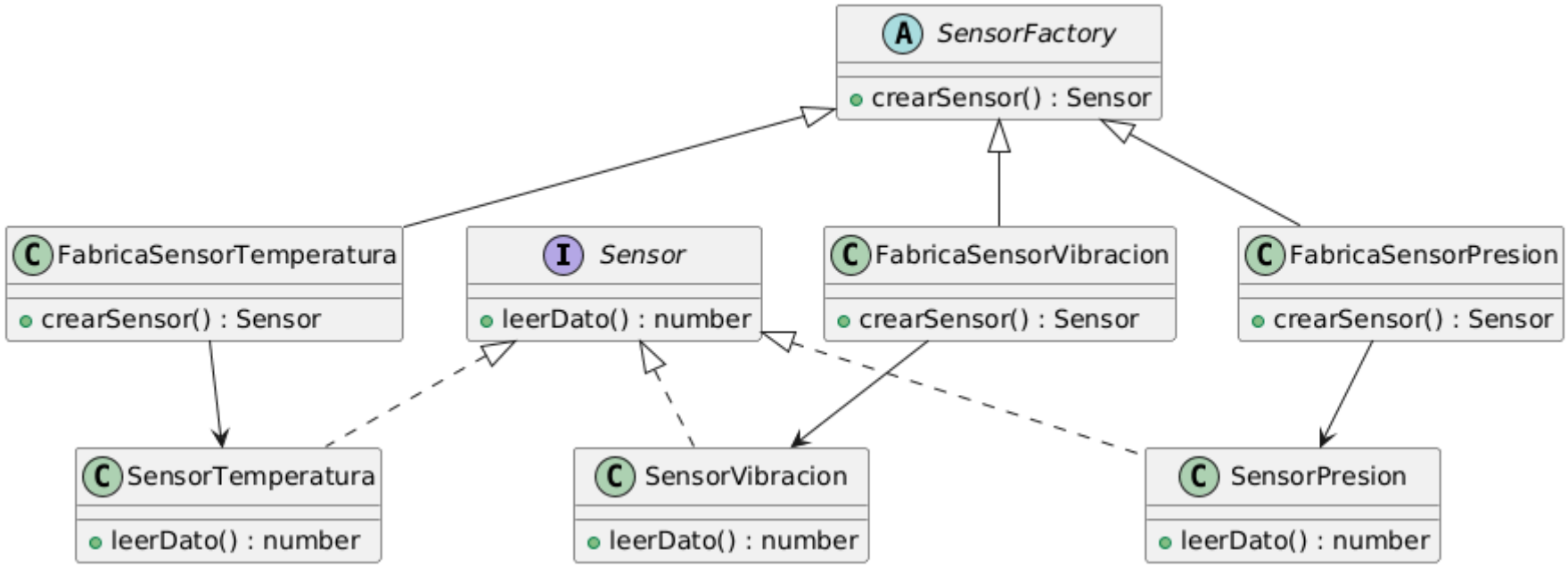
El patrón Factory Method define una **interfaz para crear objetos**, pero permite que **las subclases decidan qué tipo de objeto se debe instanciar.**

En la Plataforma Inteligente de Mantenimiento Predictivo este patrón se utiliza para **crear sensores dinámicamente según el tipo solicitado**, sin que el resto del sistema conozca la implementación específica de cada sensor.

Esto proporciona:

- Bajo acoplamiento entre clases.
- Mayor extensibilidad del sistema.
- Facilidad para agregar nuevos tipos de sensores.
- Mejor mantenimiento del código.

Diagrama UML del Patrón Factory Method



Este diagrama muestra que la **fábrica es responsable de crear los sensores**, mientras que el resto del sistema solo interactúa con la interfaz general `Sensor`.

Implementación del Patrón Factory Method

Se creó una interfaz que define el comportamiento común de todos los sensores.

Archivo:

src/sensors/interfaces/sensor.interface.ts

```
src > sensors > interfaces > TS sensor.interface.ts > Sensor
1  export interface Sensor {
2
3    leerDato(): number;
4
5  }
```

Implementación de Sensores Concretos

Se implementaron tres sensores concretos dentro del sistema.

Sensor de Temperatura

src/sensors/types/sensor-temperatura.ts

```
src > sensors > types > TS sensor-temperatura.ts > SensorTemperatura
1  import { Sensor } from "../interfaces/sensor.interface";
2
3  export class SensorTemperatura implements Sensor {
4
5    leerDato(): number {
6
7      return 75;
8
9    }
10
11 }
```

Sensor de Vibración

src/sensors/types/sensor-vibracion.ts

```
src > sensors > types > TS sensor-vibracion.ts > SensorVibracion
1   import { Sensor } from "../interfaces/sensor.interface";
2
3   export class SensorVibracion implements Sensor {
4
5     leerDato(): number {
6
7       return 40;
8
9     }
10
11  }
```

Sensor de Presión

src/sensors/types/sensor-presion.ts

```
src > sensors > types > TS sensor-presion.ts > SensorPresion
1   import { Sensor } from "../interfaces/sensor.interface";
2
3   export class SensorPresion implements Sensor {
4
5     leerDato(): number {
6
7       return 60;
8
9     }
10
11  }
```

Cada una de estas clases implementa la interfaz Sensor, permitiendo que el sistema pueda utilizarlas de manera uniforme.

Creación de la Fábrica Abstracta

Posteriormente se creó una clase abstracta que define el método que todas las fábricas deben implementar.

src/sensors/factories/sensor.factory.ts

```
src > sensors > factories > TS sensor.factory.ts > SensorFactory
1  import { Sensor } from "../interfaces/sensor.interface";
2
3  export abstract class SensorFactory {
4
5    abstract crearSensor(): Sensor;
6
7  }
```

Fábricas Concretas

Cada tipo de sensor tiene su propia fábrica encargada de crear instancias del sensor correspondiente.

Fábrica Sensor Temperatura

src > sensors > factories > TS fabrica-sensor-temperatura.ts > FabricaSensorTemperatura

```typescript
import { SensorFactory } from "./sensor.factory";
import { Sensor } from "../interfaces/sensor.interface";
import { SensorTemperatura } from "../types/sensor-temperatura";

export class FabricaSensorTemperatura extends SensorFactory {

  crearSensor(): Sensor {

    return new SensorTemperatura();

  }

}
```

Fábrica Sensor Vibración

src > sensors > factories > TS fabrica-sensor-vibracion.ts > FabricaSensorVibracion

```typescript
import { SensorFactory } from "./sensor.factory";
import { Sensor } from "../interfaces/sensor.interface";
import { SensorVibracion } from "../types/sensor-vibracion";

export class FabricaSensorVibracion extends SensorFactory {

  crearSensor(): Sensor {

    return new SensorVibracion();

  }

}
```

Fábrica Sensor Presión

src > sensors > factories > TS fabrica-sensor-presion.ts > FabricaSensorPresion

```
1   import { SensorFactory } from "./sensor.factory";
2   import { Sensor } from "../interfaces/sensor.interface";
3   import { SensorPresion } from "../types/sensor-presion";
4
5   export class FabricaSensorPresion extends SensorFactory {
6
7     crearSensor(): Sensor {
8
9       return new SensorPresion();
10
11    }
12
13  }
```

Cada fábrica concreta es responsable de crear el sensor específico que representa.

Integración del patrón en el sistema

Para utilizar las fábricas dentro del sistema se implementó la lógica correspondiente dentro del servicio de sensores

src/sensors/sensors.service.ts

```
  leerSensor(tipo: string) {

    let sensor;

    if (tipo === "temperatura") {

      const fabrica = new FabricaSensorTemperatura();
      sensor = fabrica.crearSensor();

    }
    else if (tipo === "vibracion") {

      const fabrica = new FabricaSensorVibracion();
      sensor = fabrica.crearSensor();

    }
    else if (tipo === "presion") {

      const fabrica = new FabricaSensorPresion();
      sensor = fabrica.crearSensor();

    }
    else {

      return "Tipo de sensor no válido";

    }

    return sensor.leerDato();

  }

}
```

Esta lógica permite que el sistema seleccione dinámicamente la fábrica correspondiente según el tipo de sensor solicitado.

Endpoint para probar el patrón

Para probar el funcionamiento del patrón se creó un endpoint dentro del controlador de sensores.

src/sensors/sensors.controller.ts

```
// ENDPOINT DEL FACTORY METHOD
@Post("leer")
leer(@Body() body:any){

  return this.sensorsService.leerSensor(body.tipo);

}

}
```

Este endpoint permite solicitar la lectura de un sensor específico mediante una petición HTTP.

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado:

POST http://localhost:3000/sensors/leer

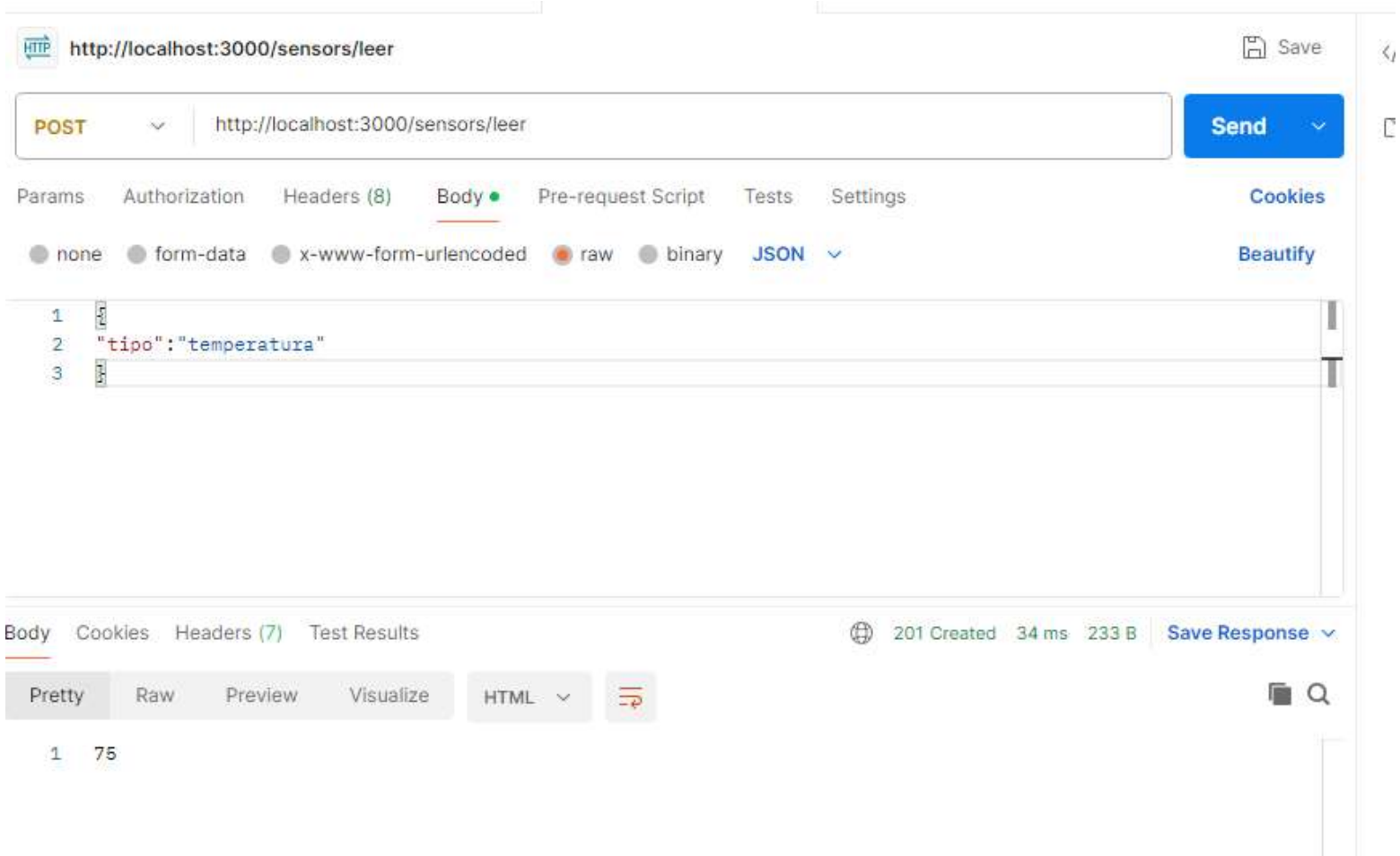
Prueba 1 — Sensor de temperatura

Petición enviada:

{

"tipo":"temperatura"

}



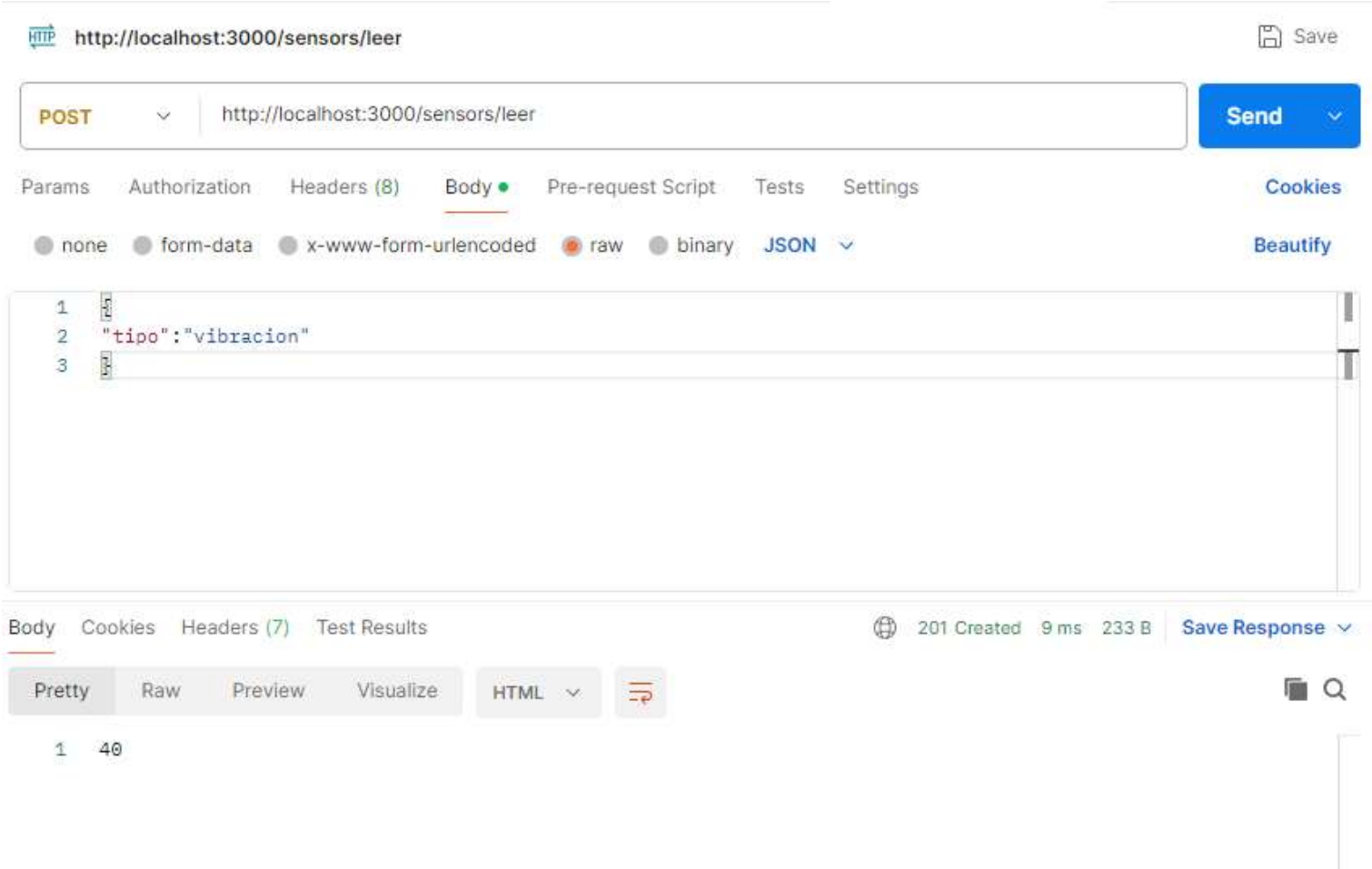
Prueba 2 — Sensor de vibración

Petición enviada:

{

"tipo":"vibracion"

}



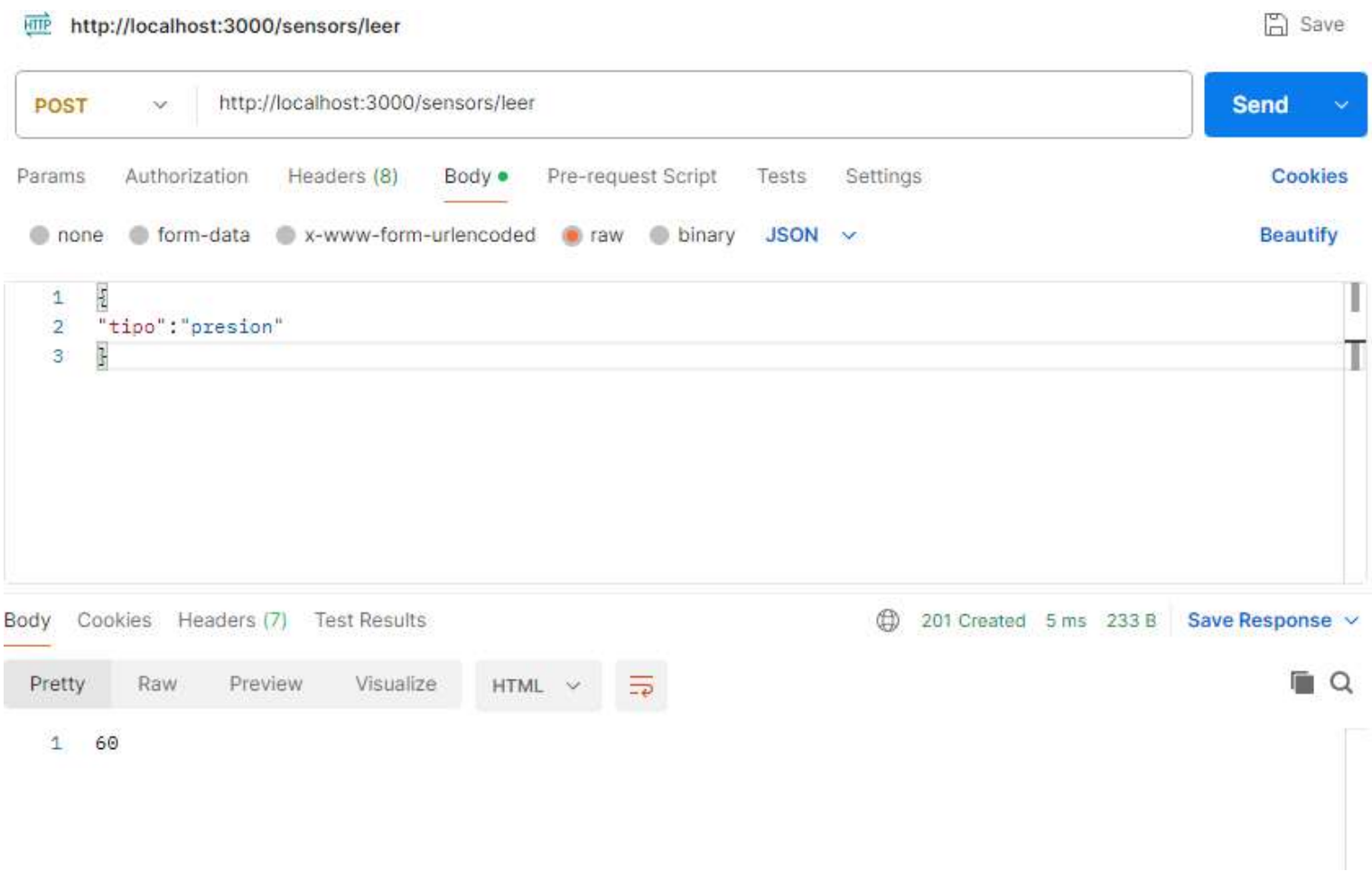
Prueba 3 — Sensor de presión

Petición enviada:

{

"tipo":"presion"

}



Flujo de funcionamiento

El flujo de ejecución del sistema es el siguiente:

Cliente

↓

API REST

↓

SensorsController

↓

SensorsService

↓

Factory Method

↓

SensorConcreto

El servicio selecciona la fábrica adecuada y esta se encarga de crear el sensor correspondiente.

Patrón Abstract Factory

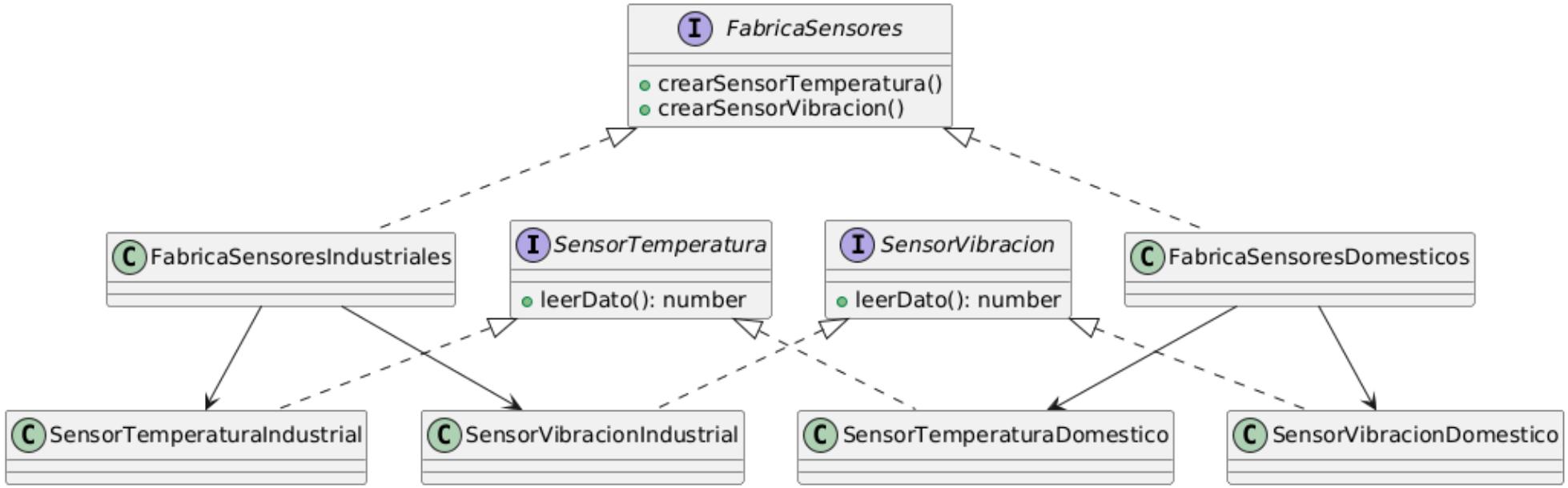
El patrón Abstract Factory proporciona una interfaz para crear **familias de objetos relacionados o dependientes sin especificar sus clases concretas.**

En la Plataforma Inteligente de Mantenimiento Predictivo este patrón se utiliza para **crear familias completas de sensores**, permitiendo que el sistema seleccione automáticamente el tipo adecuado dependiendo del entorno de operación.

Las ventajas principales de este patrón son:

- Permite crear **familias completas de objetos relacionados**.
- Reduce el acoplamiento entre clases concretas.
- Facilita la expansión del sistema.
- Permite cambiar configuraciones completas del sistema sin modificar la lógica principal.

Diagrama UML del Patrón Abstract Factory



Este diagrama muestra cómo una fábrica abstracta puede generar **diferentes familias de sensores.**

Implementación en el sistema

Para implementar el patrón se crearon primero las interfaces de los sensores, que definen los métodos que todos los sensores deben implementar.

Posteriormente se desarrollaron las clases concretas de sensores industriales y domésticos, las cuales representan implementaciones específicas para cada tipo de entorno.

Después se definió la fábrica abstracta de sensores, la cual contiene los métodos necesarios para crear sensores de temperatura y vibración.

Finalmente se crearon las fábricas concretas, que son las responsables de generar sensores pertenecientes a una familia específica.

De esta manera, el sistema puede crear sensores industriales o domésticos dependiendo del tipo de fábrica que se utilice.

Interfaz de Sensores de Temperatura

src/sensors/abstract/temperatura.interface.ts

```
src > sensors > abstract > TS temperatura.interface.ts > SensorTemperatura
1  export interface SensorTemperatura {
2
3    leerDato(): number;
4
5  }
```

Este método representa la lectura del sensor de temperatura y devuelve un valor numérico.

Todas las implementaciones de sensores de temperatura deben implementar este método.

Interfaz de Sensores de Vibración

src/sensors/abstract/vibracion.interface.ts

src > sensors > abstract > TS vibracion.interface.ts > SensorVibracion

```
export interface SensorVibracion {

  leerDato(): number;

}
```

Implementación de sensores domésticos

Los sensores domésticos representan dispositivos utilizados en entornos de menor exigencia.

Sensor temperatura doméstico

src/sensors/domestico/temperatura-domestico.ts

src > sensors > domestico > TS temperatura-domestico.ts > SensorTemperaturaDomestico

```
import { SensorTemperatura } from "../abstract/temperatura.interface";

export class SensorTemperaturaDomestico implements SensorTemperatura {

  leerDato(): number {

    return 35;

  }

}
```

Este sensor representa un sensor de temperatura utilizado en ambientes domésticos.

El método leerDato() retorna 35 grados, que representa una temperatura típica de ambiente.

Sensor vibración doméstico

src/sensors/domestico/vibracion-domestico.ts

Este sensor devuelve un valor de 10, simulando una vibración baja típica de un entorno doméstico.

Definición de la fábrica abstracta

La fábrica abstracta define qué tipos de productos puede crear cada familia de sensores.

src/sensors/abstract/sensor-abstract.factory.ts

src > sensors > abstract > TS sensor-abstract.factory.ts > FabricaSensores

```
1   import { SensorTemperatura } from "./temperatura.interface";
2   import { SensorVibracion } from "./vibracion.interface";
3
4   export interface FabricaSensores {
5
6     crearSensorTemperatura(): SensorTemperatura;
7
8     crearSensorVibracion(): SensorVibracion;
9
10  }
```

Esta interfaz define dos métodos:

crearSensorTemperatura()

crearSensorVibracion()

Esto significa que todas las fábricas concretas deberán crear ambos tipos de sensores.

Fábrica de sensores industriales

src/sensors/factories/fabrica-industrial.ts

src > sensors > factories > TS fabrica-domestica.ts > FabricaSensoresDomesticos

```
1  import { FabricaSensores } from "../abstract/sensor-abstract.factory";
2  import { SensorTemperatura } from "../abstract/temperatura.interface";
3  import { SensorVibracion } from "../abstract/vibracion.interface";
4
5  import { SensorTemperaturaDomestico } from "../domestico/temperatura-domestico";
6  import { SensorVibracionDomestico } from "../domestico/vibracion-domestico";
7
8  export class FabricaSensoresDomesticos implements FabricaSensores {
9
10   crearSensorTemperatura(): SensorTemperatura {
11
12     return new SensorTemperaturaDomestico();
13
14   }
15
16   crearSensorVibracion(): SensorVibracion {
17
18     return new SensorVibracionDomestico();
19
20   }
21
22 }
```

Esta fábrica crea sensores industriales.

Cuando el sistema solicita:

crearSensorTemperatura()

se crea un:

SensorTemperaturaIndustrial

Cuando solicita:

crearSensorVibracion()

se crea un:

SensorVibracionIndustrial

Fábrica de sensores domésticos

src/sensors/factories/fabrica-domestica.ts

src > sensors > factories > TS fabrica-domestica.ts > FabricaSensoresDomesticos

```
1  import { FabricaSensores } from "../abstract/sensor-abstract.factory";
2  import { SensorTemperatura } from "../abstract/temperatura.interface";
3  import { SensorVibracion } from "../abstract/vibracion.interface";
4
5  import { SensorTemperaturaDomestico } from "../domestico/temperatura-domestico";
6  import { SensorVibracionDomestico } from "../domestico/vibracion-domestico";
7
8  export class FabricaSensoresDomesticos implements FabricaSensores {
9
10   crearSensorTemperatura(): SensorTemperatura {
11
12     return new SensorTemperaturaDomestico();
13
14   }
15
16   crearSensorVibracion(): SensorVibracion {
17
18     return new SensorVibracionDomestico();
19
20   }
21
22 }
```

Esta fábrica genera sensores diseñados para ambientes domésticos.

Cuando el sistema utiliza esta fábrica se crean sensores compatibles con ese entorno.

Integración en el servicio del sistema

src/sensors/sensors.service.ts

```
// ========================
// PATRÓN ABSTRACT FACTORY
// ========================

crearFamiliaSensores(tipo: string) {

  let fabrica;

  if (tipo === "industrial") {

    fabrica = new FabricaSensoresIndustriales();

  }
  else {

    fabrica = new FabricaSensoresDomesticos();

  }

  const sensorTemp = fabrica.crearSensorTemperatura();
  const sensorVib = fabrica.crearSensorVibracion();

  return {

    temperatura: sensorTemp.leerDato(),
    vibracion: sensorVib.leerDato()

  };

}

}
```

Este método permite seleccionar dinámicamente la fábrica que se utilizará.

Si el tipo es:

Este método permite seleccionar dinámicamente la fábrica que se utilizará.

Si el tipo es:

industrial

se crea una fábrica de sensores industriales. Si es de domestico se crea una fábrica de sensores domésticos.

Endpoint del controlador

src/sensors/sensors.controller.ts

```
//ENDPOINT ABSTRACT METHOD
@Post("familia")
crearFamilia(@Body() body:any){

return this.sensorsService.crearFamiliaSensores(body.tipo);

}

}
```

Este endpoint permite probar el patrón Abstract Factory mediante una petición HTTP.

El usuario envía el tipo de familia de sensores y el sistema crea los sensores correspondientes.

Pruebas del sistema

Endpoint

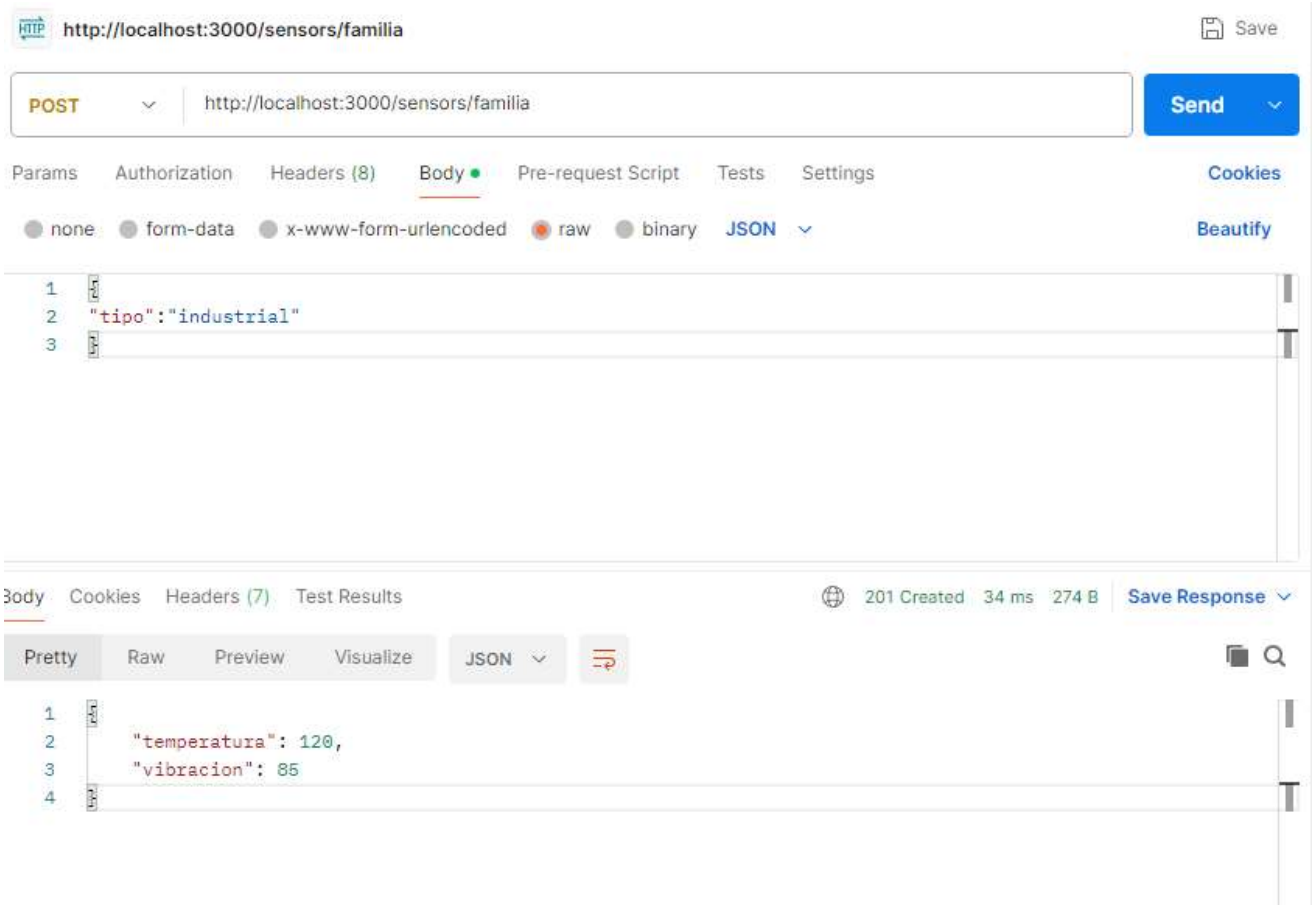
POST http://localhost:3000/sensors/familia

Prueba industrial

{

"tipo":"industrial"

}

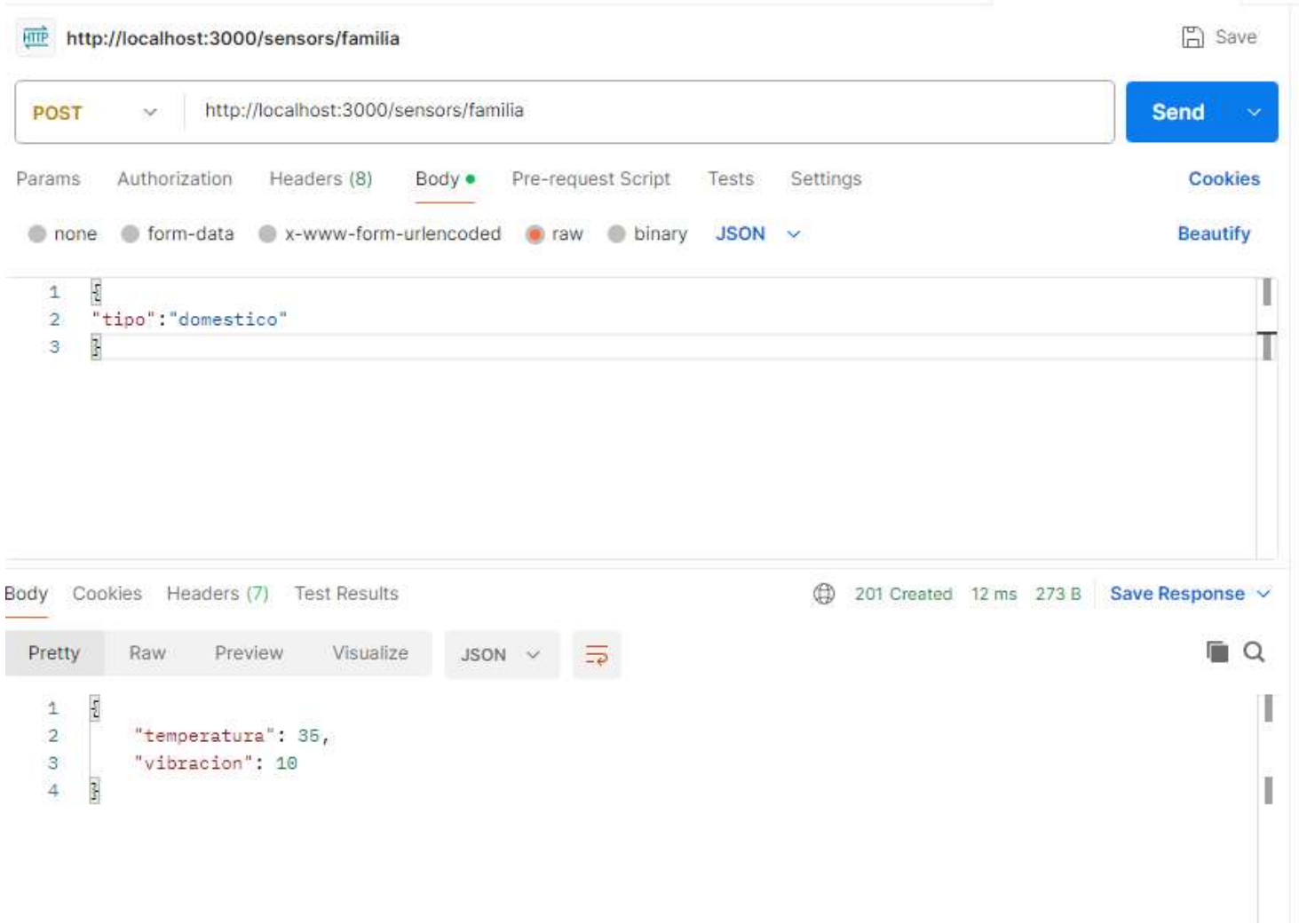


Prueba doméstica

{

"tipo":"domestico"

}



Patrón Builder

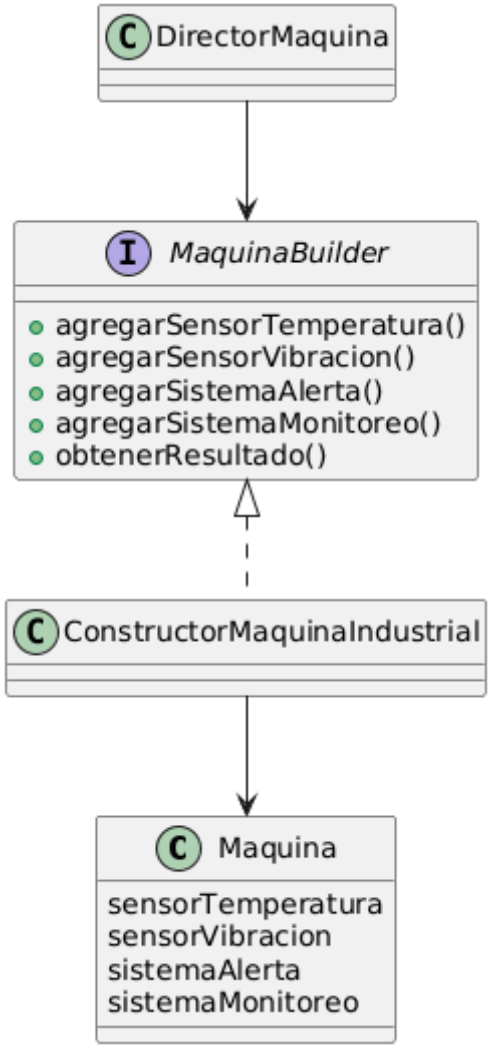
El patrón Builder permite construir objetos complejos paso a paso, separando el proceso de construcción de la representación final del objeto. Esto permite crear diferentes configuraciones del mismo objeto utilizando el mismo proceso de construcción.

En la Plataforma Inteligente de Mantenimiento Predictivo este patrón se utiliza para construir **configuraciones de máquinas industriales**, las cuales pueden incluir distintos tipos de sensores y sistemas de monitoreo.

Entre las ventajas de utilizar este patrón se encuentran:

- Permite construir objetos complejos paso a paso.
- Mejora la legibilidad del código.
- Facilita la creación de diferentes configuraciones del mismo objeto.
- Reduce el acoplamiento entre el proceso de construcción y el objeto final.

Diagrama UML del patrón Builder



Este diagrama muestra cómo el **director controla la construcción**, mientras que el **Builder construye el objeto paso a paso.**

Clase del objeto complejo

src/machines/maquina.ts

```
src > machines > TS maquina.ts > Maquina
1   export class Maquina {
2
3   sensorTemperatura: boolean = false;
4   sensorVibracion: boolean = false;
5   sistemaAlerta: boolean = false;
6   sistemaMonitoreo: boolean = false;
7
8   }
```

La clase **Maquina** representa una máquina industrial que puede tener diferentes componentes:

- Sensor de temperatura
- Sensor de vibración
- Sistema de alerta
- Sistema de monitoreo

Estos componentes se activan durante el proceso de construcción.

Interfaz Builder

src/machines/maquina.builder.ts

```
src > machines > TS maquina.builder.ts > MaquinaBuilder
1   import { Maquina } from "./maquina";
2
3   export interface MaquinaBuilder {
4
5   agregarSensorTemperatura(): void;
6
7   agregarSensorVibracion(): void;
8
9   agregarSistemaAlerta(): void;
10
11  agregarSistemaMonitoreo(): void;
12
13  obtenerResultado(): Maquina;
14
15  }
```

Esta interfaz define los pasos necesarios para construir una máquina. Cada implementación concreta del Builder deberá implementar estos métodos.

Builder concreto

src/machines/constructor-maquina-industrial.ts

```
src > machines > constructor-maquina-industrial.ts > ConstructorMaquinaIndustrial > agregarSensorTemperatura
1   import { Maquina } from "./maquina";
2   import { MaquinaBuilder } from "./maquina.builder";
3
4   export class ConstructorMaquinaIndustrial implements MaquinaBuilder {
5
6   private maquina: Maquina;
7
8   constructor(){
9
10  this.maquina = new Maquina();
11
12  }
13
14  agregarSensorTemperatura(): void {
15
16  this.maquina.sensorTemperatura = true;
17
18  }
19
20  agregarSensorVibracion(): void {
21
22  this.maquina.sensorVibracion = true;
23
24  }
25
26  agregarSistemaAlerta(): void {
27
28  this.maquina.sistemaAlerta = true;
29
30  }
31
32  agregarSistemaMonitoreo(): void {
33
34  this.maquina.sistemaMonitoreo = true;
35
36  }
37
```

```
}

obtenerResultado(): Maquina {

return this.maquina;

}

}
```

Esta clase implementa el proceso de construcción de una máquina industrial.

Cada método activa uno de los componentes de la máquina.

Director de construcción

src/machines/director-maquina.ts

```
src > machines > TS director-maquina.ts > DirectorMaquina
1   import { MaquinaBuilder } from "./maquina.builder";
2
3   export class DirectorMaquina {
4
5   construirMaquinaCompleta(builder: MaquinaBuilder){
6
7   builder.agregarSensorTemperatura();
8   builder.agregarSensorVibracion();
9   builder.agregarSistemaAlerta();
10  builder.agregarSistemaMonitoreo();
11
12  }
13
14  }
```

El **Director** controla el proceso de construcción.

Define el orden en el que se agregan los componentes de la máquina.

Integración en el servicio

src/sensors/sensors.service.ts

```
import { ConstructorMaquinaIndustrial } from '../machines/constructor-maquina-industrial';
import { DirectorMaquina } from '../machines/director-maquina';
```

```
// ========================
// PATRÓN BUILDER
// ========================

construirMaquina() {

  const builder = new ConstructorMaquinaIndustrial();

  const director = new DirectorMaquina();

  director.construirMaquinaCompleta(builder);

  return builder.obtenerResultado();

}

}
```

Este método crea una máquina industrial utilizando el patrón Builder.

Primero se crea el Builder, luego el Director controla el proceso de construcción.

Finalmente se obtiene el objeto máquina construido.

Endpoint para probar el patrón

```
@Post("maquina")
crearMaquina(){

return this.sensorsService.construirMaquina();

}
```

Pruebas del sistema

POST http://localhost:3000/sensors/maquina

Unidades Tecnológicas de Santander

http://localhost:3000/sensors/maquina

Save

POST http://localhost:3000/sensors/maquina Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

ody Cookies Headers (7) Test Results 201 Created 14 ms 334 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "sensorTemperatura": true,
    "sensorVibracion": true,
    "sistemaAlerta": true,
    "sistemaMonitoreo": true
}
```

Flujo de funcionamiento

Cliente

↓

Controller

↓

SensorsService

↓

DirectorMaquina

↓

Builder

↓

Objeto Maquina

La implementación del patrón Builder permitió crear configuraciones complejas de máquinas industriales de manera organizada y flexible.

Este patrón separa el proceso de construcción del objeto final, permitiendo que diferentes configuraciones puedan generarse utilizando el mismo proceso.

Además, mejora la legibilidad del código y facilita la expansión futura del sistema.

Durante esta primera etapa del proyecto se diseñó e implementó el backend utilizando un enfoque modular que permite separar claramente las responsabilidades del sistema. Para lograr una arquitectura flexible, mantenible y escalable, se implementaron diferentes patrones de diseño que permiten gestionar de manera eficiente la creación y administración de los componentes del sistema.

En primer lugar, se implementó el patrón Singleton, el cual permite garantizar que el motor de análisis predictivo tenga una única instancia dentro del sistema. Esto asegura un control centralizado del proceso de análisis de datos provenientes de los sensores.

Posteriormente se implementó el patrón Factory Method, el cual permite crear dinámicamente diferentes tipos de sensores, como sensores de temperatura, vibración y presión, sin que el sistema dependa directamente de sus clases concretas. Esto facilita la incorporación de nuevos sensores en el futuro sin modificar la lógica principal de la aplicación.

Adicionalmente se implementó el patrón Abstract Factory, el cual permite crear familias completas de sensores según el tipo de entorno en el que se utilicen, por ejemplo sensores industriales o sensores domésticos. Este patrón mejora la organización del sistema y garantiza la compatibilidad entre los componentes creados.

Finalmente se integró el patrón Builder, el cual permite construir configuraciones complejas de máquinas industriales paso a paso. Este enfoque facilita la creación de diferentes configuraciones del sistema de monitoreo sin alterar la estructura del código.

La implementación de estos patrones de diseño permite establecer una base arquitectónica robusta para el desarrollo del proyecto, facilitando su evolución en futuras etapas. Sin embargo, es importante destacar que el sistema aún se encuentra en una fase inicial de desarrollo y todavía faltan múltiples componentes para convertirlo en una solución completa lista para producción.

CORTE 2

Patrón Prototype

El patrón Prototype permite crear nuevos objetos clonando una instancia existente, en lugar de construirlos desde cero.

Esto resulta útil cuando los objetos tienen configuraciones complejas o cuando se desea reutilizar una estructura base sin repetir código.

Aplicación en nuestro sistema

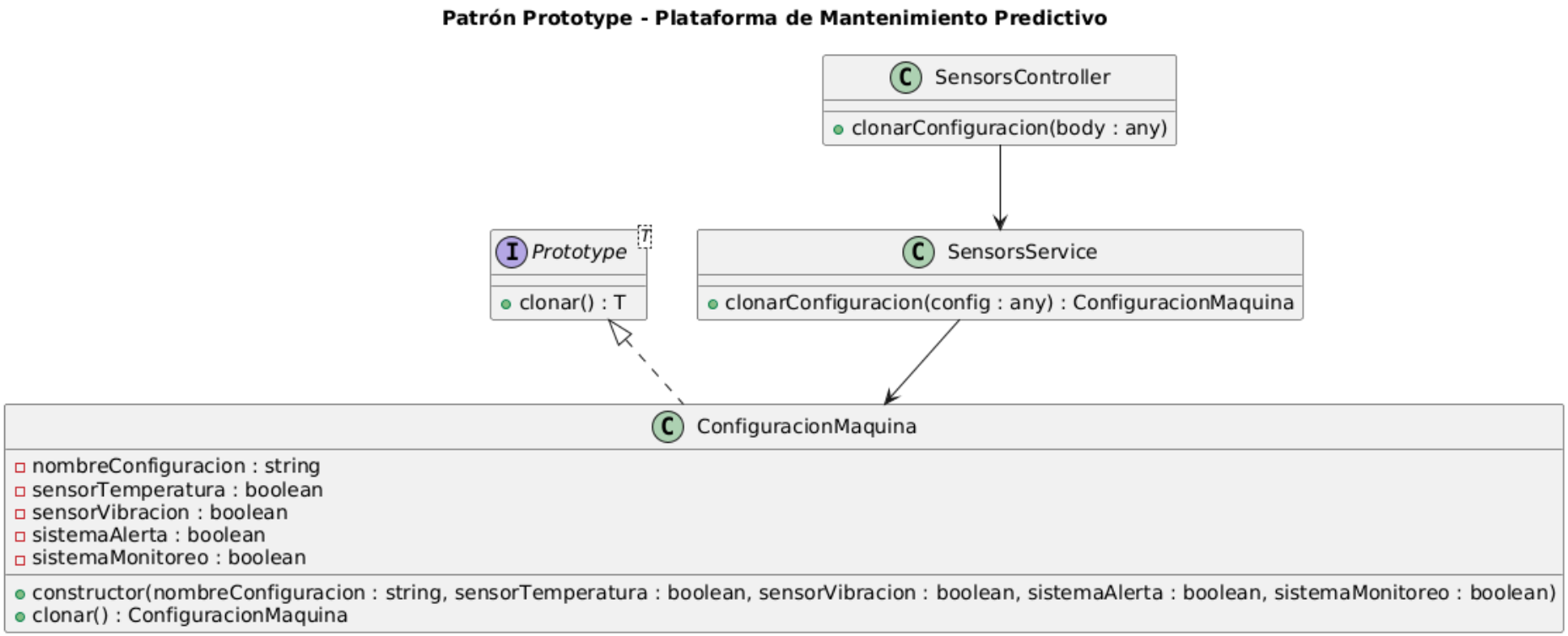
En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Prototype se utilizará para clonar configuraciones de máquinas industriales.

Esto permite:

- reutilizar configuraciones ya definidas
- crear rápidamente nuevas máquinas con la misma estructura base
- reducir código repetido
- facilitar la personalización de configuraciones

Por ejemplo, si ya existe una máquina configurada con sensor de temperatura, vibración, sistema de alerta y sistema de monitoreo, el sistema puede clonar esa configuración y modificar solo algunos atributos según la necesidad.

Diagrama UML del Patrón Prototype



Implementación del Patrón Prototype

Se creó una interfaz que define el comportamiento general de clonación.

Archivo

src/machines/prototype/prototype.interface.ts

```
TS prototype.interface.ts U ×
src > machines > prototype > TS prototype.interface.ts > ...
1  export interface Prototype<T> {
2    clonar(): T;
3  }
4
```

Clase concreta que implementa Prototype

Se implementó una clase que representa una configuración reutilizable de máquina industrial.

Archivo

src/machines/prototype/configuracion-maquina.prototype.ts

```
src > machines > prototype > TS configuracion-maquina.prototype.ts > ConfiguracionMaquina
1  import { Prototype } from './prototype.interface';
2
3  export class ConfiguracionMaquina implements Prototype<ConfiguracionMaquina> {
4    constructor(
5      public nombreConfiguracion: string,
6      public sensorTemperatura: boolean,
7      public sensorVibracion: boolean,
8      public sistemaAlerta: boolean,
9      public sistemaMonitoreo: boolean,
10   ) {}
11
12   clonar(): ConfiguracionMaquina {
13     return new ConfiguracionMaquina(
14       this.nombreConfiguracion,
15       this.sensorTemperatura,
16       this.sensorVibracion,
17       this.sistemaAlerta,
18       this.sistemaMonitoreo,
19     );
20   }
21 }
```

Integración del patrón en el sistema

Para utilizar el patrón Prototype dentro del sistema, se implementó un método en el servicio de sensores que recibe una configuración base, la clona y permite modificar el nombre de la nueva configuración.

Archivo

src/sensors/sensors.service.ts

```
// ========================
// PATRÓN PROTOTYPE
// ========================

clonarConfiguracion(body: any) {
  const configuracionBase = new ConfiguracionMaquina(
    body.nombreConfiguracion,
    body.sensorTemperatura,
    body.sensorVibracion,
    body.sistemaAlerta,
    body.sistemaMonitoreo
  );

  const clon = configuracionBase.clonar();
  clon.nombreConfiguracion = body.nuevoNombre;

  return clon;
}
```

Endpoint para probar el patrón

Se creó un endpoint en el controlador para clonar configuraciones de máquina mediante una petición HTTP.

```
// ENDPOINT PROTOTYPE
@Post("clonar-configuracion")
clonarConfiguracion(@Body() body: any) {

  return this.sensorsService.clonarConfiguracion(body);

}
```

Pruebas del sistema del patrón Prototype

Las pruebas del patrón Prototype se realizaron utilizando Postman para enviar solicitudes HTTP al backend del sistema.

Endpoint utilizado

POST http://localhost:3000/sensors/clonar-configuracion

Prueba del sistema

Para probar el funcionamiento del patrón se envió una petición HTTP con una configuración base de máquina industrial.

El sistema recibe la configuración, crea el objeto original y posteriormente genera una copia utilizando el método de clonación.

Petición enviada

POST http://localhost:3000/sensors/clonar-configuracion Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2   "nombreConfiguracion": "Maquina Base Industrial",
3   "sensorTemperatura": true,
4   "sensorVibracion": true,
5   "sistemaAlerta": true,
6   "sistemaMonitoreo": true,
7   "nuevoNombre": "Maquina Clonada Industrial"
8 }
```

Respuesta obtenida

Body Cookies Headers (7) Test Results 201 Created 342 ms 386 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "nombreConfiguracion": "Maquina Clonada Industrial",
    "sensorTemperatura": true,
    "sensorVibracion": true,
    "sistemaAlerta": true,
    "sistemaMonitoreo": true
}
```


Explicación del resultado

La respuesta obtenida demuestra que el sistema clonó correctamente la configuración original de la máquina.

La nueva configuración conserva todos los atributos del objeto base:

- sensor de temperatura
- sensor de vibración
- sistema de alerta
- sistema de monitoreo

Sin embargo, el nombre de la configuración fue modificado, lo que confirma que se creó una nueva instancia a partir del objeto original.

Esto demuestra el funcionamiento correcto del patrón Prototype dentro del sistema.

Patrón Adapter

El patrón Adapter permite que dos clases con interfaces incompatibles puedan trabajar juntas.

Actúa como un intermediario que transforma la interfaz de una clase existente en otra que el sistema espera utilizar.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Adapter se utilizará para integrar sensores externos o heredados cuyos datos no vienen en el mismo formato que maneja internamente el sistema.

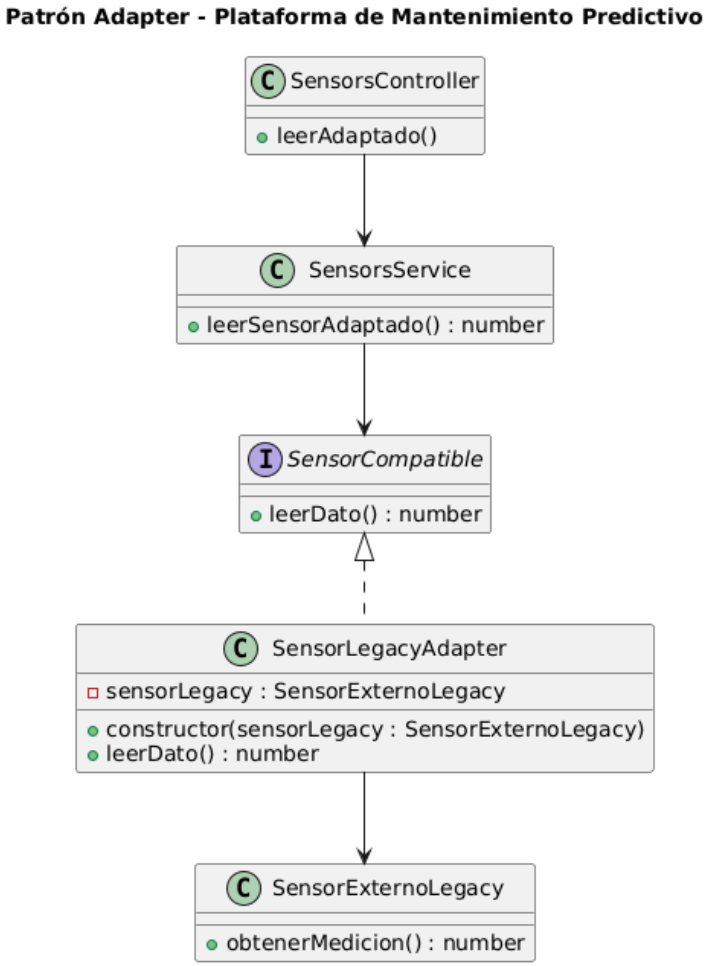
Esto permite:

- integrar dispositivos externos sin modificar la lógica principal
- reutilizar sensores legacy ya existentes
- mantener desacoplado el sistema interno de proveedores externos
- estandarizar los datos que recibe el backend

Por ejemplo, un sensor externo puede entregar los datos mediante un método llamado obtenerMedicion(), mientras que el sistema trabaja con sensores que implementan el método leerDato().

El Adapter permite convertir ese sensor externo al formato esperado por la plataforma.

Diagrama UML



Implementación del Patrón Adapter

Primero se creó una interfaz compatible con el sistema actual.

Archivo

src/sensors/adapters/sensor-compatible.interface.ts

```
src > sensors > adapters > TS sensor-compatible.interface.ts > SensorCompatible
1  export interface SensorCompatible {
2    leerDato(): number;
3  }
```

Implementación del sensor externo heredado

Se creó una clase que simula un sensor externo o legacy.
Este sensor no implementa la interfaz del sistema y utiliza un método diferente para obtener la medición.

```
src > sensors > adapters > TS sensor-legacy.ts > SensorExternoLegacy
1  export class SensorExternoLegacy {
2    obtenerMedicion(): number {
3      return 92;
4    }
5  }
```

Implementación del adaptador

Se implementó una clase adaptadora que recibe el sensor externo heredado y transforma su método al formato esperado por el sistema.

Archivo

src/sensors/adapters/sensor-legacy.adapter.ts

src > sensors > adapters > sensor-legacy.adapter.ts > SensorLegacyAdapter

```
1   import { SensorCompatible } from './sensor-compatible.interface';
2   import { SensorExternoLegacy } from './sensor-legacy';
3
4   export class SensorLegacyAdapter implements SensorCompatible {
5     constructor(private readonly sensorLegacy: SensorExternoLegacy) {}
6
7     leerDato(): number {
8       return this.sensorLegacy.obtenerMedicion();
9     }
10  }
```

Integración del patrón en el sistema

Para integrar este patrón dentro del sistema, se implementó un método en el servicio de sensores que crea el sensor legacy, lo adapta y retorna su lectura.

Archivo

src/sensors/sensors.service.ts

```
leerSensorAdaptado() {

  const sensorLegacy = new SensorExternoLegacy();

  const adaptador = new SensorLegacyAdapter(sensorLegacy);

  return adaptador.leerDato();

}
```

Endpoint del controlador

Para probar el funcionamiento del patrón Adapter se creó un endpoint en el controlador de sensores

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT ADAPTER
@Post("adaptado")
leerAdaptado() {

  return this.sensorsService.leerSensorAdaptado();

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

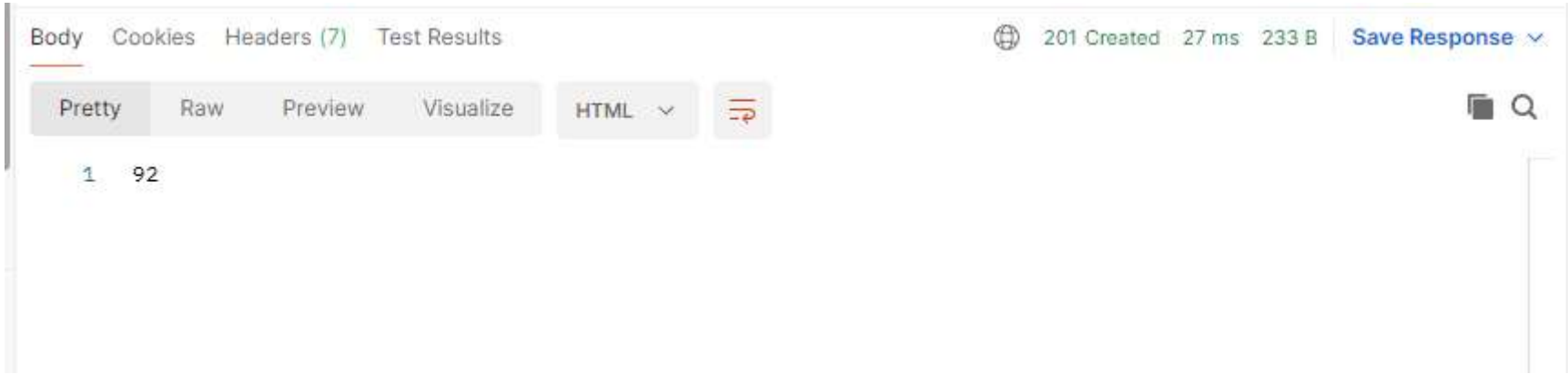
POST http://localhost:3000/sensors/adaptado

Prueba del sistema

Para probar el patrón Adapter se envió una petición HTTP al endpoint correspondiente.
En este caso no es necesario enviar datos en el body, ya que el sistema internamente crea el sensor externo y utiliza el adaptador para transformar su interfaz.

Petición enviada

No requiere body.



Patrón Bridge

El patrón Bridge permite separar una abstracción de su implementación, de modo que ambas puedan variar de manera independiente.

Este patrón resulta útil cuando un sistema necesita combinar diferentes tipos de objetos con distintas formas de ejecución, evitando una explosión de clases y reduciendo el acoplamiento entre componentes.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Bridge se utilizará para desacoplar el tipo de alerta de su canal de notificación.

Esto significa que el sistema puede manejar diferentes tipos de alertas, por ejemplo:

• alerta crítica

• alerta preventiva

y enviarlas a través de diferentes canales, por ejemplo:

• correo electrónico

• panel del sistema

De esta manera, el sistema puede combinar libremente cada tipo de alerta con cualquier canal de notificación sin depender de clases concretas.

Ventajas de aplicar este patrón

• Permite desacoplar la lógica de la alerta de la forma en que se notifica.

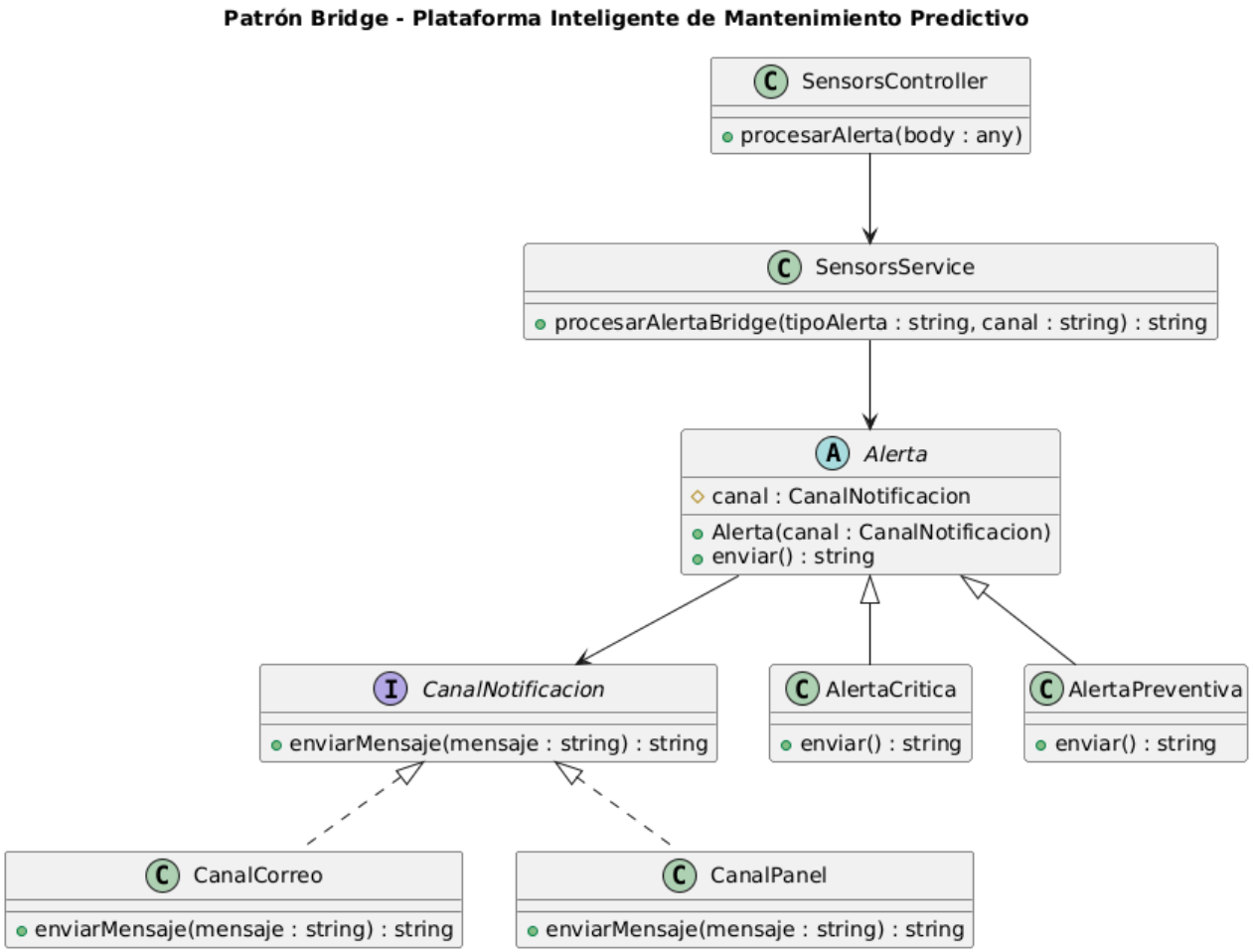
• Facilita agregar nuevos tipos de alertas.

• Facilita agregar nuevos canales de notificación.

• Mejora la escalabilidad y mantenibilidad del sistema.

• Reduce el acoplamiento entre componentes del backend

Diagrama UML del patrón Bridge



Implementación del patrón Bridge

Primero se creó la interfaz que representa la implementación del canal de notificación.

Archivo

src/alerts/bridge/canal-notificacion.interface.ts

```
TS canal-notificacion.interface.ts U ×
src > alerts > bridge > TS canal-notificacion.interface.ts > CanalNotificacion
1  export interface CanalNotificacion {
2    enviarMensaje(mensaje: string): string;
3  }
```

Implementaciones concretas de canal

Se implementaron dos canales concretos para enviar alertas dentro del sistema.

src/alerts/bridge/canal-correo.ts

```
src > alerts > bridge > TS canal-correo.ts > CanalCorreo
1  import { CanalNotificacion } from './canal-notificacion.interface';
2
3  export class CanalCorreo implements CanalNotificacion {
4    enviarMensaje(mensaje: string): string {
5      return `Correo enviado con mensaje: ${mensaje}`;
6    }
7  }
```

Canal del panel del sistema

src/alerts/bridge/canal-panel.ts

Abstracción principal

Posteriormente se creó la abstracción Alerta, la cual trabaja con cualquier implementación de canal de notificación.

Archivo

src/alerts/bridge/alerta.abstract.ts

```
src > alerts > bridge > TS alerta.abstract.ts > Alerta
1  import { CanalNotificacion } from './canal-notificacion.interface';
2
3  export abstract class Alerta {
4    constructor(protected canal: CanalNotificacion) {}
5
6    abstract enviar(): string;
7  }
```

Alertas concretas

Se implementaron dos tipos de alertas dentro del sistema.

Alerta crítica

src/alerts/bridge/alerta-critica.ts

```
src > alerts > bridge > TS alerta-critica.ts > AlertaCritica
1  import { Alerta } from './alerta.abstract';
2  import { CanalNotificacion } from './canal-notificacion.interface';
3
4  export class AlertaCritica extends Alerta {
5    constructor(canal: CanalNotificacion) {
6      super(canal);
7    }
8
9    enviar(): string {
10     return this.canal.enviarMensaje('ALERTA CRÍTICA: posible falla grave detectada');
11   }
12 }
```

Alerta preventiva

src/alerts/bridge/alerta-preventiva.ts

```
src > alerts > bridge > TS alerta-preventiva.ts > AlertaPreventiva
1  import { Alerta } from './alerta.abstract';
2  import { CanalNotificacion } from './canal-notificacion.interface';
3
4  export class AlertaPreventiva extends Alerta {
5    constructor(canal: CanalNotificacion) {
6      super(canal);
7    }
8
9    enviar(): string {
10     return this.canal.enviarMensaje('ALERTA PREVENTIVA: revisar comportamiento del activo');
11   }
12 }
```

Integración del patrón en el sistema

Para integrar el patrón Bridge se implementó un método en el servicio de sensores que recibe el tipo de alerta y el canal deseado, construye los objetos adecuados y ejecuta el envío.

Archivo

src/sensors/sensors.service.ts

```
//PATRON BRIDGE

procesarAlertaBridge(tipoAlerta: string, canal: string) {
  let canalNotificacion;
  let alerta;

  if (canal === 'correo') {
    canalNotificacion = new CanalCorreo();
  } else {
    canalNotificacion = new CanalPanel();
  }

  if (tipoAlerta === 'critica') {
    alerta = new AlertaCritica(canalNotificacion);
  } else {
    alerta = new AlertaPreventiva(canalNotificacion);
  }

  return alerta.enviar();
}
```

Endpoint del controlador

Para probar el patrón Bridge se creó un endpoint que permite seleccionar el tipo de alerta y el canal de notificación.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT BRIDGE
@Post("alerta-bridge")
procesarAlerta(@Body() body: any) {

  return this.sensorsService.procesarAlertaBridge(body.tipoAlerta, body.canal);

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/alerta-bridge

Prueba 1 — Alerta crítica por correo

Petición enviada

none JSON Text

```
{
  "tipoAlerta": "critica",
  "canal": "correo"
}
```

Respuesta

```
"Correo enviado con mensaje: ALERTA CRÍTICA: posible falla grave detectada"
```

Prueba 2 — Alerta preventiva en panel

Petición enviada

none JSON Text

```
{
  "tipoAlerta": "preventiva",
  "canal": "panel"
}
```

Respuesta

```
"Notificación mostrada en panel: ALERTA PREVENTIVA: revisar comportamiento del activo"
```

Esto confirma que la abstracción Alerta y su implementación CanalNotificacion están desacopladas, tal como lo propone el patrón Bridge.

Patrón Decorator

El patrón Decorator permite agregar nuevas responsabilidades o comportamientos a un objeto de manera dinámica, sin modificar su estructura original.

Este patrón resulta útil cuando se desea extender la funcionalidad de un componente base de forma flexible y reutilizable.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Decorator se utilizará para enriquecer el procesamiento de lecturas de sensores.

En este caso, una lectura base de sensor puede ser decorada con funcionalidades adicionales, por ejemplo:

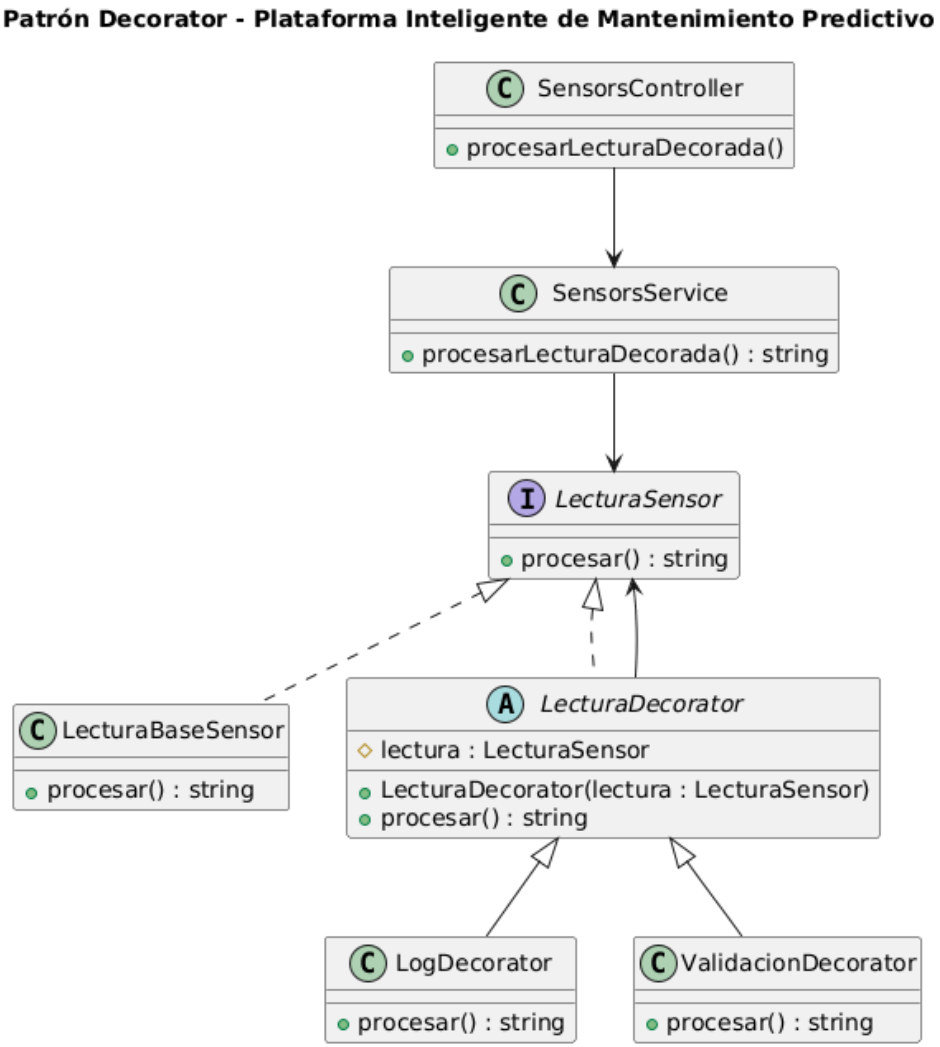
- registro en log
- validación de rangos
- marcado de prioridad

Esto permite ampliar la funcionalidad del procesamiento sin modificar la clase original del sensor.

Ventajas de aplicar este patrón

- Permite extender funcionalidades sin alterar el objeto base.
- Favorece el principio de abierto/cerrado.
- Facilita combinar múltiples comportamientos adicionales.
- Reduce el acoplamiento y mejora la reutilización del código.
- Permite construir flujos de procesamiento más flexibles.

Diagrama UML del patrón Decorator



Implementación del patrón Decorator

Se creó primero una interfaz común para representar el procesamiento de lecturas de sensores.

src > sensors > decorator > TS lectura-sensor.interface.ts > LecturaSensor

```
1  export interface LecturaSensor {
2    procesar(): string;
3  }
```

Componente base

Se implementó una clase base que representa el procesamiento simple de una lectura de sensor.

Archivo

src/sensors/decorator/lectura-base-sensor.ts

src > sensors > decorator > TS lectura-base-sensor.ts > LecturaBaseSensor

```
1  import { LecturaSensor } from './lectura-sensor.interface';
2
3  export class LecturaBaseSensor implements LecturaSensor {
4    procesar(): string {
5      return 'Lectura base del sensor procesada';
6    }
7  }
```

Decorador abstracto

Luego se implementó un decorador abstracto que mantiene la referencia al componente base.

Archivo

src/sensors/decorator/lectura.decorator.ts

src > sensors > decorator > TS lectura.decorator.ts > LecturaDecorator

```
1  import { LecturaSensor } from './lectura-sensor.interface';
2
3  export abstract class LecturaDecorator implements LecturaSensor {
4    constructor(protected lectura: LecturaSensor) {}
5
6    procesar(): string {
7      return this.lectura.procesar();
8    }
9  }
```

Decoradores concretos

Se implementaron decoradores que agregan nuevas funcionalidades al procesamiento de la lectura.

Decorador de log

src/sensors/decorator/log.decorator.ts

src > sensors > decorator > TS log.decorator.ts > LogDecorator

```typescript
import { LecturaDecorator } from './lectura.decorator';
import { LecturaSensor } from './lectura-sensor.interface';

export class LogDecorator extends LecturaDecorator {
  constructor(lectura: LecturaSensor) {
    super(lectura);
  }

  procesar(): string {
    return `${super.procesar()} | Registro en log completado`;
  }
}
```

Decorador de validación

src/sensors/decorator/validacion.decorator.ts

src > sensors > decorator > TS validacion.decorator.ts > ValidacionDecorator

```typescript
import { LecturaDecorator } from './lectura.decorator';
import { LecturaSensor } from './lectura-sensor.interface';

export class ValidacionDecorator extends LecturaDecorator {
  constructor(lectura: LecturaSensor) {
    super(lectura);
  }

  procesar(): string {
    return `${super.procesar()} | Validación de rango ejecutada`;
  }
}
```

Integración del patrón en el sistema

Para integrar el patrón Decorator se creó un método en el servicio de sensores que construye una lectura base y la decora con comportamientos adicionales.

Archivo

src/sensors/sensors.service.ts

```
//PATRON DECORATOR
procesarLecturaDecorada() {
  let lectura = new LecturaBaseSensor();

  lectura = new LogDecorator(lectura);
  lectura = new ValidacionDecorator(lectura);

  return lectura.procesar();
}
```

Endpoint del controlador

Para probar el patrón Decorator se creó un endpoint que ejecuta el procesamiento decorado de una lectura de sensor.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT DECORATOR
@Post("lectura-decorada")
procesarLecturaDecorada() {

  return this.sensorsService.procesarLecturaDecorada();

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/lectura-decorada

Prueba del sistema

Para probar el patrón Decorator se envió una petición HTTP al endpoint correspondiente.

En este caso no es necesario enviar información en el body, ya que el sistema internamente construye la lectura base y le agrega los decoradores definidos.

Petición enviada

No requiere body.

Respuesta

```
"Lectura base del sensor procesada | Registro en log completado | Validación de rango ejecutada"
```

La respuesta demuestra que el sistema primero ejecutó el procesamiento base y luego fue agregando dinámicamente nuevas responsabilidades mediante los decoradores.

La implementación del patrón Decorator permite agregar funcionalidades al procesamiento de lecturas de sensores sin modificar la estructura base del sistema.

Patrón Composite

El patrón Composite permite tratar objetos individuales y composiciones de objetos de manera uniforme.

Este patrón resulta útil cuando se trabaja con estructuras jerárquicas, donde un objeto puede estar compuesto por varios elementos del mismo tipo.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Composite se utilizará para representar grupos de activos industriales.

Esto permite que el sistema pueda manejar:

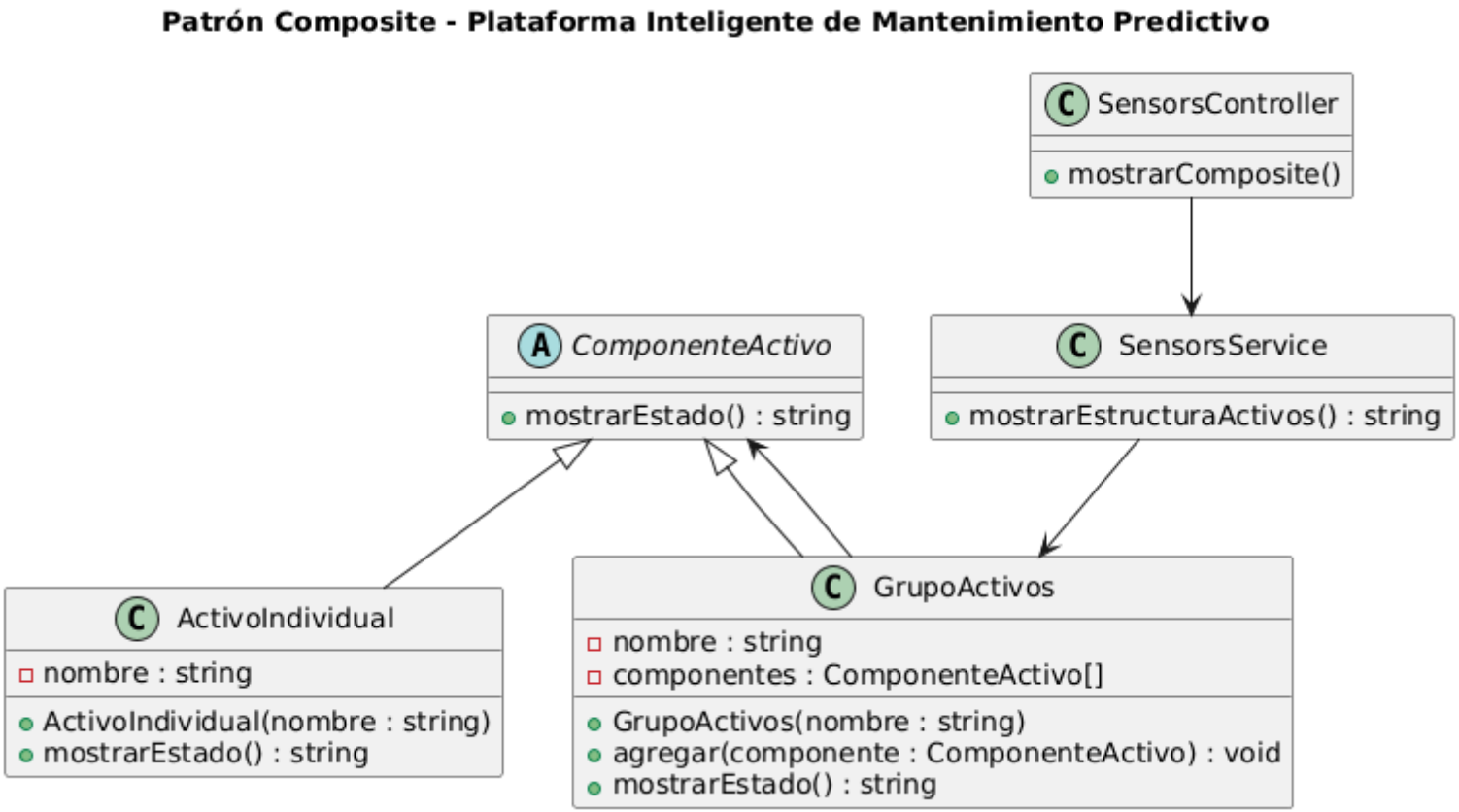
- un activo individual
- un conjunto de activos
- una planta compuesta por varias máquinas
- una línea de producción compuesta por varios grupos de activos

De esta manera, el sistema puede calcular el estado o mostrar la estructura de activos de forma uniforme, sin importar si se trata de un solo equipo o de un grupo completo.

Ventajas de aplicar este patrón

- Permite modelar estructuras jerárquicas de activos.
- Facilita tratar un activo simple y un grupo de activos de la misma forma.
- Reduce la complejidad al recorrer estructuras compuestas.
- Facilita la escalabilidad del sistema.
- Mejora la organización del módulo de activos industriales.

Diagrama UML del patrón Composite



Implementación del patrón Composite

Primero se creó una clase abstracta que representa el comportamiento común de cualquier activo dentro del sistema.

Archivo

src/assets/composite/componente-activo.ts

```
TS componente-activo.ts U ×
src > assets > composite > TS componente-activo.ts > ...
export abstract class ComponenteActivo {
  abstract mostrarEstado(): string;
}
```

Implementación de activo individual

Se implementó una clase concreta que representa un activo individual dentro de la plataforma.

Archivo

src/assets/composite/activo-individual.ts

```
src > assets > composite > TS activo-individual.ts > ActivoIndividual
import { ComponenteActivo } from './componente-activo';

export class ActivoIndividual extends ComponenteActivo {
  constructor(private nombre: string) {
    super();
  }

  mostrarEstado(): string {
    return `Activo individual: ${this.nombre}`;
  }
}
```

Implementación del grupo de activos

Posteriormente se implementó una clase compuesta que puede contener varios activos o incluso otros grupos de activos.

Archivo

src/assets/composite/grupo-activos.ts

src > assets > composite > TS grupo-activos.ts > GrupoActivos

```typescript
import { ComponenteActivo } from './componente-activo';

export class GrupoActivos extends ComponenteActivo {
  private componentes: ComponenteActivo[] = [];

  constructor(private nombre: string) {
    super();
  }

  agregar(componente: ComponenteActivo): void {
    this.componentes.push(componente);
  }

  mostrarEstado(): string {
    let resultado = `Grupo: ${this.nombre}\n`;

    for (const componente of this.componentes) {
      resultado += componente.mostrarEstado() + '\n';
    }

    return resultado.trim();
  }
}
```

Integración del patrón en el sistema

Para integrar el patrón Composite en el sistema, se implementó un método en el servicio que crea activos individuales y grupos compuestos, devolviendo la estructura completa.

Archivo

src/sensors/sensors.service.ts

```
mostrarEstructuraActivos() {
  const activo1 = new ActivoIndividual('Motor Principal');
  const activo2 = new ActivoIndividual('Bomba Hidráulica');
  const activo3 = new ActivoIndividual('Compresor Secundario');

  const grupoProduccion = new GrupoActivos('Línea de Producción'
  grupoProduccion.agregar(activo1);
  grupoProduccion.agregar(activo2);

  const planta = new GrupoActivos('Planta Industrial');
  planta.agregar(grupoProduccion);
  planta.agregar(activo3);

  return planta.mostrarEstado();
}
```

Endpoint del controlador

Para probar el funcionamiento del patrón Composite se creó un endpoint en el controlador.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT COMPOSITE
@Post("composite")
mostrarComposite() {

  return this.sensorsService.mostrarEstructuraActivos();

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/composite

Prueba del sistema

Para probar el patrón Composite se envió una petición HTTP al endpoint correspondiente.

En este caso no es necesario enviar body, ya que el sistema crea internamente la estructura jerárquica de activos.

Petición enviada

No requiere body.

Respuesta obtenida

Body Cookies Headers (7) Test Results

Pretty Raw Preview Visualize HTML

```
1  Grupo: Planta Industrial
2  Grupo: Línea de Producción
3  Activo individual: Motor Principal
4  Activo individual: Bomba Hidráulica
5  Activo individual: Compresor Secundario
```

Explicación del resultado

La respuesta demuestra que el sistema logró representar una estructura jerárquica compuesta por activos individuales y grupos de activos.

Esto confirma que el patrón Composite está funcionando correctamente, ya que el sistema trata de manera uniforme tanto a los elementos simples como a los compuestos.

Flujo de funcionamiento del patrón Composite

El flujo del proceso implementado es el siguiente:

Cliente

↓

API REST

↓

SensorsController

↓

SensorsService

↓

GrupoActivos

↓

ActivoIndividual / GrupoActivos

↓

Resultado final

Patrón Facade

El patrón Facade proporciona una interfaz simplificada para acceder a un conjunto de subsistemas complejos.

Este patrón permite ocultar la complejidad interna del sistema y ofrecer un punto de acceso más claro y sencillo para el cliente.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Facade se utilizará para centralizar el proceso de monitoreo de un activo industrial.

Actualmente, para monitorear un activo podrían intervenir varios procesos internos, por ejemplo:

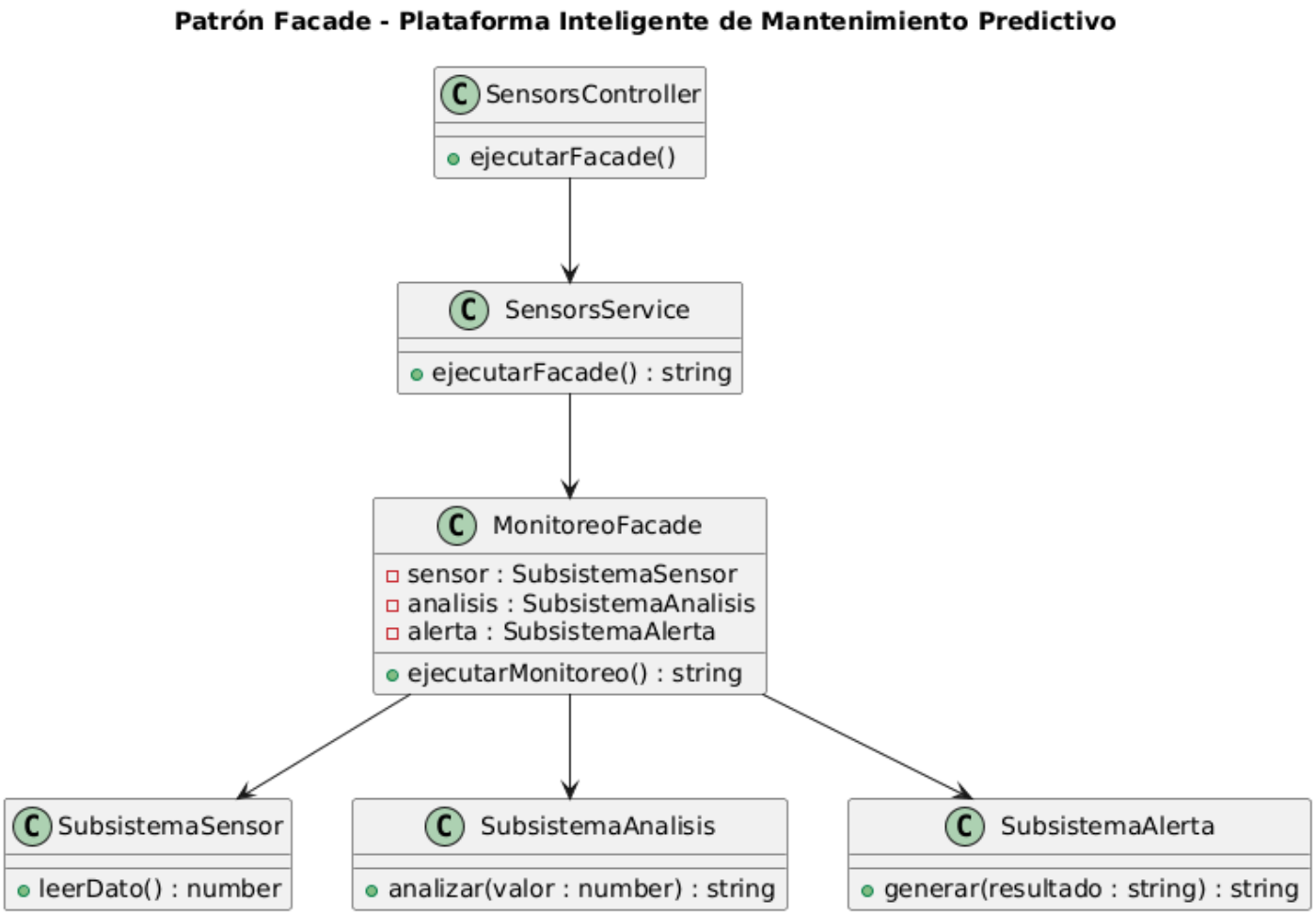
- lectura de sensores
- análisis del valor recibido
- generación de alerta
- construcción del resultado final

Con el patrón Facade, todos estos pasos se agrupan en una sola clase que ofrece una operación simple al resto del sistema.

Ventajas de aplicar este patrón

- Simplifica el acceso a subsistemas complejos.
- Reduce el acoplamiento entre el cliente y la lógica interna.
- Mejora la legibilidad del código.
- Facilita el mantenimiento del sistema.
- Permite centralizar procesos repetitivos del backend.

Diagrama UML del patrón Facade



Implementación del patrón Facade

Primero se crearon los subsistemas que participan en el proceso de monitoreo.

Subsistema de sensor

Archivo

src/monitoring/facade/subsistema-sensor.ts

```
src > monitoring > facade > TS subsistema-sensor.ts > SubsistemaSensor
1  export class SubsistemaSensor {
2    leerDato(): number {
3      return 85;
4    }
5  }
```

Subsistema de análisis

Archivo

src/monitoring/facade/subsistema-analisis.ts

```
src > monitoring > facade > TS subsistema-analisis.ts > ...
1  export class SubsistemaAnalisis {
2    analizar(valor: number): string {
3      if (valor > 80) {
4        return 'Anomalía detectada';
5      }
6
7      return 'Funcionamiento normal';
8    }
9  }
10
```

Subsistema de alerta

Archivo

src/monitoring/facade/subsistema-alerta.ts

```
src > monitoring > facade > TS subsistema-alerta.ts > SubsistemaAlerta
1  export class SubsistemaAlerta {
2    generar(resultado: string): string {
3      if (resultado === 'Anomalía detectada') {
4        return 'ALERTA: posible falla detectada';
5      }
6
7      return 'Sin alertas';
8    }
9  }
```

Implementación de la fachada

Posteriormente se creó una clase fachada que coordina la interacción entre los subsistemas.

Archivo

src/monitoring/facade/monitoreo.facade.ts

src > monitoring > facade > TS monitoreo.facade.ts > MonitoreoFacade

```typescript
import { SubsistemaSensor } from './subsistema-sensor';
import { SubsistemaAnalisis } from './subsistema-analisis';
import { SubsistemaAlerta } from './subsistema-alerta';

export class MonitoreoFacade {
  private sensor: SubsistemaSensor;
  private analisis: SubsistemaAnalisis;
  private alerta: SubsistemaAlerta;

  constructor() {
    this.sensor = new SubsistemaSensor();
    this.analisis = new SubsistemaAnalisis();
    this.alerta = new SubsistemaAlerta();
  }

  ejecutarMonitoreo(): string {
    const valor = this.sensor.leerDato();
    const resultadoAnalisis = this.analisis.analizar(valor);
    const alerta = this.alerta.generar(resultadoAnalisis);

    return `Valor leído: ${valor} | Resultado: ${resultadoAnalisis} | ${alerta}`;
  }
}
```

Integración del patrón en el sistema

Para integrar el patrón Facade en el sistema, se implementó un método en el servicio que utiliza la fachada para ejecutar el proceso completo de monitoreo.

Archivo

src/sensors/sensors.service.ts

```typescript
//PATRON FACADE
ejecutarFacade() {
  const facade = new MonitoreoFacade();

  return facade.ejecutarMonitoreo();
}
```

Endpoint del controlador

Para probar el patrón Facade se creó un endpoint en el controlador.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT FACADE
@Post("facade")
ejecutarFacade() {

  return this.sensorsService.ejecutarFacade();

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/facade

Prueba del sistema

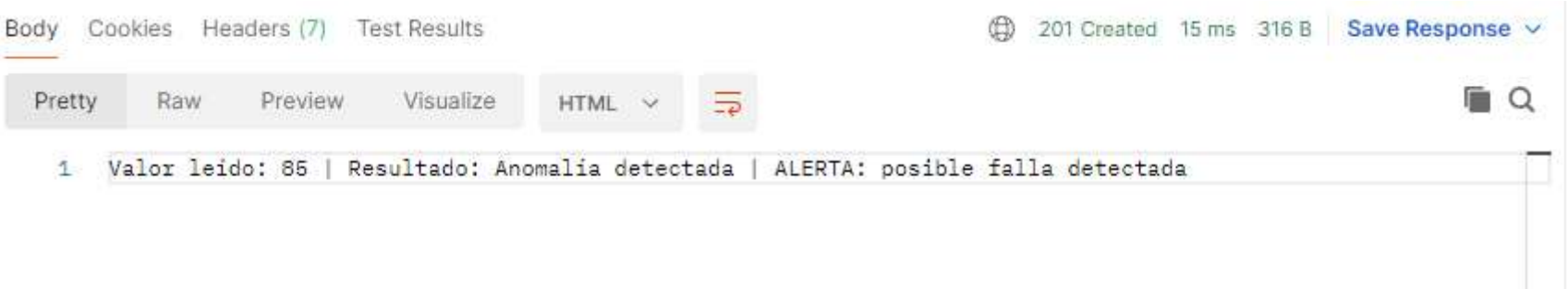
Para probar el patrón Facade se envió una petición HTTP al endpoint correspondiente.

No es necesario enviar body, ya que la fachada se encarga de coordinar internamente todos los subsistemas.

Petición enviada

No requiere body.

Respuesta obtenida



Explicación del resultado

La respuesta demuestra que el sistema ejecutó de manera centralizada varios procesos internos: lectura del sensor, análisis del valor y generación de alerta.

Esto confirma que el patrón Facade está funcionando correctamente, ya que proporciona una interfaz simple para un proceso que internamente involucra varios subsistemas.

Flujo de funcionamiento del patrón Facade

El flujo del proceso implementado es el siguiente:

Cliente

↓

API REST

↓

SensorsController

↓

SensorsService

↓

MonitoreoFacade

↓

SubsistemaSensor

↓

SubsistemaAnalisis

↓

SubsistemaAlerta

↓

Resultado final